Imperial College
London

Imperial College London – Department of Electrical & Electronic Engineering

Adaptive Signal Processing and Machine Intelligence Coursework

Mohammed Kamal Khwaja
CID: 01226910
March 30, 2017

Supervised by:
Prof. Danilo P. Mandic
Imperial College London

Overall not a bad attempt. Some sections are however left unfinished, and some explanations like the one about the frequency resolution are superficial and do not show a deep understanding of the subject. Still, this is an above average report.

Table of Contents

Good table of contents

1. Spectrum Estimation **3**
1.1. Discrete Fourier Transform Basics 3
a. Ideal Spectrum 3
b. DFT Spectra using MATLAB 3
c. Incoherent Sampling 4
1.2 Properties of Power Spectral Density (PSD) 4
Approximation in the definition of PSD 4
Using DTFT to determine PSD from ACF 5
1.3. Resolution and Leakage of Periodogram-based methods 6
1.4. Periodogram-based Methods Applied to Real-World Data 10
a. The sunspot time series 10
b. Electroencephalogram (EEG) data 11

2. Parametric and Line Spectra **13**
2.1 Correlation Estimation 13
Biased and unbiased estimators 13
2.2 Spectrum of Autoregressive Processes 16
2.3 Principal Component Analysis 17
2.4 Real World Signals: Respiratory Sinus Arrhythmia from RR-Intervals 19

3. Adaptive Signal Processing **22**
3.1 The Least Mean Square (LMS) Algorithm 22
3.2 Adaptive Step Sizes 25
3.3 Adaptive Noise Cancellation 27

4. Widely Linear Filtering and Adaptive Spectrum Estimation **31**
4.1 Complex LMS and Widely Linear Modelling 31
Frequency Estimation in Three Phase Power Systems 33
4.2 Adaptive AR Model Based Time-Frequency Estimation 35
4.3 A Real Time Spectrum Analyzer Using Least Mean Square 36

Conclusion **38**

References **39**

1. Spectrum Estimation

1.1. Discrete Fourier Transform Basics

a. Ideal Spectrum

From its definition, the Fourier Transform converts a signal in the time domain into an alternate representation in the frequency domain. Thus, the Continuous Time Fourier Transform (CTFT) seeks to break down a periodic and continuous signal like a sine wave into discrete frequency values. This is done for better mathematical representability and operation. The ideal two-sided Fourier Magnitude of a 20 Hz sine wave is represented by two impulses, one at 20 Hz and the other at -20 Hz as shown in Figure 1.1.



(a) (b)

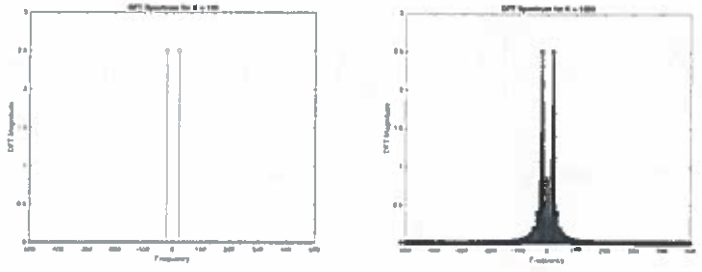
Although tidy, drawings/pictures are preferably avoidable.

Figure 1.1 (a) Ideal two-sided Fourier Magnitude of a 20 Hz sine wave; (b) DTFT of a 20 Hz sine wave

The Discrete Time Fourier Transform (DTFT) is a windowed variation of the CTFT, which attempts to restrict the duration of the signal under test and sample it periodically over the entire range of time. This is achieved by passing the signal through a band pass filter (windows), and the periodicity varies based on the size of the window. Considering a rectangular window in the time domain, the DTFT of a 20 Hz sine wave is a sequence of sinc signals. This is the result of the convolution of the rectangular window and sine wave in the time domain, which is the same as the multiplication of impulse and sinc in the frequency domain. Figure 1.1 (b) represents the DTFT of a 20 Hz sine wave by taking a two-sided rectangular window of bandwidth 20 Hz.

b. DFT Spectra using MATLAB

A 20 Hz sine wave, with an amplitude of 5, sampling frequency fs = 1 kHz and length N = 100 was generated. The 'K' DFT spectra was calculated for two cases: K = 100 and K = 1000. Figure 1.2 represents the results obtained for the two cases. The results are centered and the axis is defined to view Frequency in Hz. The peaks in both cases are as expected: at +20 Hz and -20 Hz. But the smoothness of the DFT spectrum varies distinctly in the two cases. In the case of K = 100, the frequency resolution is 10. Hence, the DFT estimates occur every 10 Hz. Since the sine wave frequency is a multiple of 10, the spectrum indicates clear peaks at the sine frequency and zeros otherwise. In thw case of K = 1000, the frequency resolution is 1. This allows for a larger resolution due to smaller frequency width. Since the original signal is of length 100, **zero padding** is performed to obtain a sequence of length 1000 and subsequently the 1000 point DFT is obtained. The zero padding results in a sinc interpolation in the frequency domain that results in a smoother spectrum, albeit with many unnecessary side lobes due to the sinc function.



(a) (b)

good use of bold terms

Figure 1.2 (a) DFT Spectrum for K = 100; (b) DFT Spectrum for K = 100

c. Incoherent Sampling

A 24 Hz sine wave, with an amplitude of 5, sampling frequency fs = 1 kHz and length N = 100 was generated. The 'K' DFT spectra was calculated for K = 100. It is seen from Figure 1.3 (a) that the peaks are not obtained at 24 Hz, but rather at 20 Hz. This is attributed to the fact that the frequency resolution, which is 10, is not able to accommodate the impulse at 24 Hz. Rather, it maps the impulse to the closest frequency, which is 20 Hz. This is incoherent sampling. To solve this problem, the time exposure K can be increased to decrease the frequency width, making frequency resolution more prominent, or replacing sampling frequency so that it is close to 1 kHz and a multiple of 24. The spectra obtained after replacing K as 250 and fs as 1200 Hz are shown in Figure 1.3 (b) and (c) respectively. It is seen that the frequency is mapped correctly to 24 Hz in both cases.

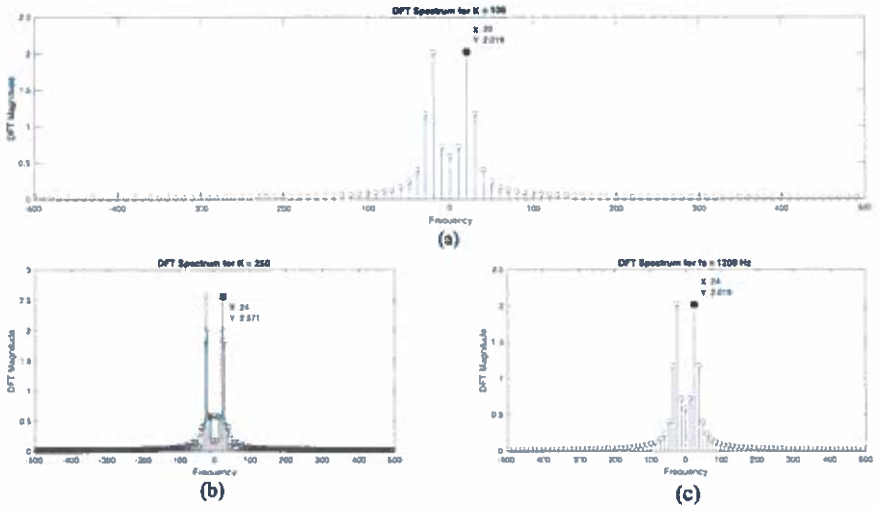


Figure 1.3 (a) DFT Spectrum for K = 100, with incoherent sampling; (b) DFT Spectrum for K = 250; (c) DFT Spectrum for f_s = 1200 Hz

1.2 Properties of Power Spectral Density (PSD)

Approximation in the definition of PSD

It is intended to prove that two definitions of PSD are equal, under the assumption:

$$\lim_{N\to\infty}\frac{1}{N}\sum_{k=-(N-1)}^{N-1}|k||r(k)| = 0 \tag{1}$$

The equivalence is proved by taking the equation and simplifying as follows:

$$\lim_{N\to\infty} E\left\{\frac{1}{N}\left|\sum_{n=0}^{N-1} y(n)\,e^{-jn\omega}\right|^2\right\} = \lim_{N\to\infty}\frac{1}{N}\sum_{n=1}^{N}\sum_{m=1}^{M} E\{y(n)y^*(m)\}\,e^{-j(n-m)\omega} \tag{2}$$

Using the double summation formula, the above is simplified to:

$$\lim_{N\to\infty}\frac{1}{N}\sum_{k=-(N-1)}^{N-1}(N-|k|)r(k)e^{-j\omega k} = \sum_{k=-\infty}^{\infty} r(k)e^{-j\omega k} - \lim_{N\to\infty}\frac{1}{N}\sum_{k=-(N-1)}^{N-1}|k|\,r(k)e^{-j\omega k} \tag{3}$$

tidy but proof is incomplete

As the second part of Equation (3) tends to zero, using the assumption in Equation (1), the remaining section of Equation (3) is the same as the original definition of PSD. Hence the equality is satisfied.

Using DTFT to determine PSD from ACF

It is verified that the DTFT can be applied to the Autocovariance Function (ACF) to obtain the PSD. For the case of length M = 10, Figure 1.4 (a) represents the PSD obtained from DTFT, while Figure 1.4 (b) represents the ACF obtained from Equation 4:

$$P(\omega) = \sum_{k=-\infty}^{\infty} r(k)e^{-j\omega k} \tag{4}$$

When M = 128, Figure 1.4 (c) represents the PSD obtained from DTFT, while Figure 1.4 (d) represents the ACF obtained from Equation 3. It is seen that the same values are obtained in both cases and the plots look similar.

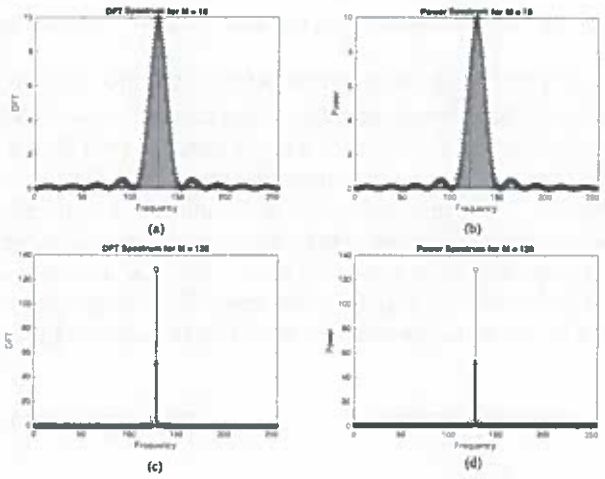
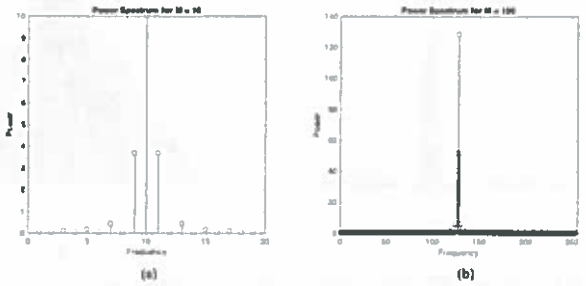


Figure 1.4 (a) DFT Spectrum for M = 10; (b) Power Spectrum for M = 10, (c) DFT Spectrum for M = 128; (d) Power Spectrum for M = 128;

a. When zeros are not padded, the results obtained are represented in Figure 1.5. It is seen from the figures that there is a difference in the resolution of the PSD for M = 10, while for the case of M = 128, the PSD remains similar to the previous case. Hence, it can be concluded that zero padding improves the frequency resolution of the DTFT, which in turns helps in distinguishing each component better.



Incorrect justification for zero padding (does not increase freq resolution)

Figure 1.5 (a) Power Spectrum for M = 10; (b) Power Spectrum for M = 128

b. Figure 1.5 also represents the real spectrum estimates obtained from the non zero padded versions of **x**. As the imaginary components of each value of the power spectrum are very small, the difference in the spectral estimate is not visible, and remains the same as the previous case.

c. The DTFT of the new vector z is shown in Figure 1.6 (a) and (b) for the case of M = 10 and M = 128 respectively. In the case of M = 10, there is a significant error as compared to the previous results. This is attributed to the fact that removing the imaginary component of the FFT, which is more significant in this case, causes the loss of phase which distorts the spectrum. In the case of M = 128, there is no padding of zeros, leading to the similar results as in the previous case.

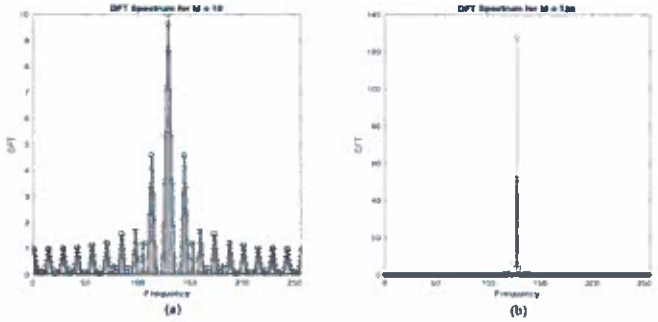


Figure 1.6 (a) DFT Spectrum for M = 10 for variation of zero padded signal; (b) DFT Spectrum for M = 128 for same signal

d. The `fftshift` command in MATLAB helps to center the ACF around the zero lag. After the use of `fftshift` was studied, all the results obtained in the previous steps have been centered around the zero lag to have an accurate analysis. By varying the k variable from -127 to 127 and taking ω from (2*π*k) /L, the frequency values are placed correctly for accurate estimation. Figure 1.7 (c) and (d) represent the Inverse DTFT for the DTFT sequences given in Figure 1.7 (a) and (b) respectively. The same input ACF is obtained, but centered at the frequency. The result depends on whether the input is odd or even. In case of odd, the input is centered at the origin, with equal values to the left and right of the x axis. In case of an even sequence, there is an imbalance of length of input vector in either side of the origin, resulting in a deviation (or lag) from the zero. This causes an error in analysis. This can be rectified by shifting the sequence by the same amount of lag, so that the center of the sequence corresponds to the center of the plot.

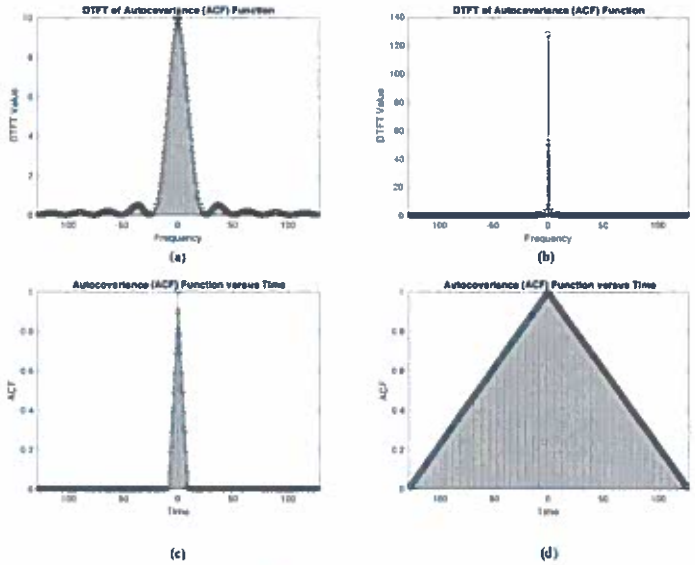
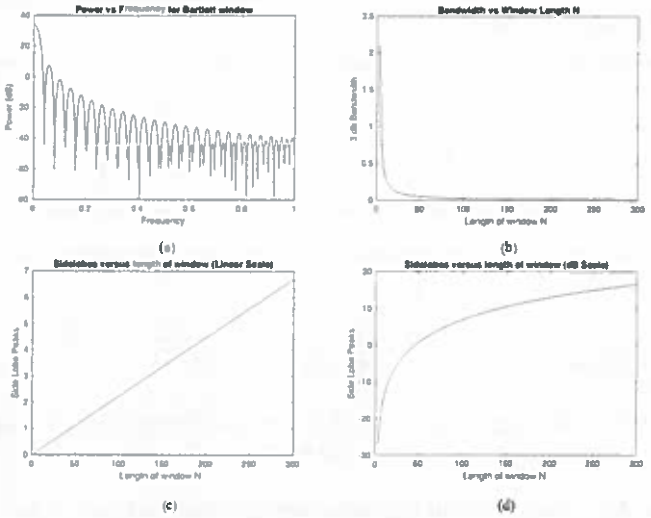


Figure 1.7 (a) DFT Spectrum for M = 10; (b) DFT Spectrum for M = 128; (c) ACF function obtained from Inverse DFT for M = 10; (d) ACF function obtained from Inverse DFT for M = 128

1.3. Resolution and Leakage of Periodogram-based methods

a. **Bartlett window:** The Bartlett window in the Frequency domain is seen in Figure 1.8 (a). The graph of the 3 dB bandwidths of the Bartlett window is seen in Figure 1.8 (b). It is confirmed that the bandwidths vary as a function

of 1/N, for increasing N values. The sidelobes are also calculated for the windows for varying N values. It is observed that the peaks vary linearly for a linear scale, given by Figure 1.8 (c). For the dB scale, it is seen that the peaks increase quickly for the first few values of N, and then increase slowly in a logarithmic fashion. This is shown in Figure 1.8 (d). It is also seen that the sidelobe peaks are approximately -27 dB from the main lobe peaks for all values of N.



As before, images are too small.

Figure 1.8 (a) Power vs Frequency for Bartlett window of size 100; (b) 3 db Bandwidth vs window length N; (c) Sidelobe peak power vs window length N; (d) Sidelobe peak power (dB) vs window length N

b. **Rectangular windowed periodogram for varying values of α:** The zero padded periodograms of two values of α are seen in Figure 1.9 (a) - (c). The minimum value of α can be calculated by considering the frequency resolution needed to distinguish the two frequencies of the sine wave. We know that the difference of the two frequencies of the sine wave should be greater than f_S/N. By substituting the values of both the frequencies and using an inequality expression, we get:

$$(f_0 + \alpha/N) - (f_0) > 2\pi f_0/N \tag{5}$$

Decent attempt at justifying frequency resolution

Simplifying,

$$\alpha > 2\pi f_0 \tag{6}$$

Substituting $f_0 = 0.2$ in Equation (6) indicates that $\alpha > 1.256$. Hence for all values of $\alpha > 1.256$, the frequency can be resolved. This in indicated in Figure 1.9 where a value of $\alpha = 1.4$ can be used to resolve the two frequencies, whereas a value of $\alpha = 1.4$ does not succeed in this motive.

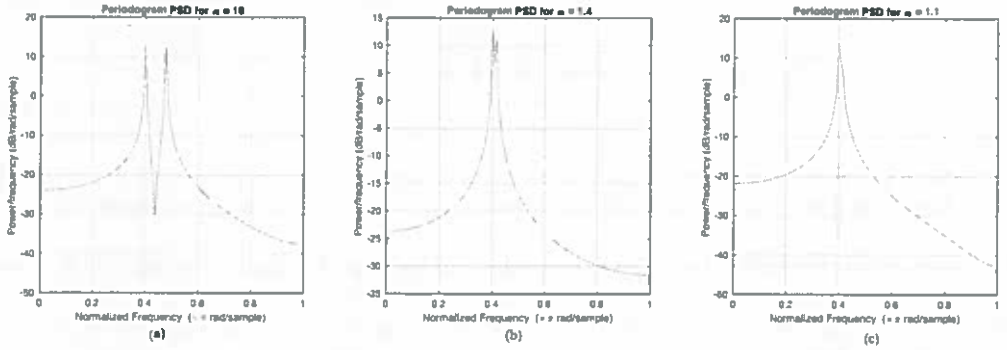


Figure 1.9 (a) Rectangular windowed periodogram for α = 10; (b) Rectangular windowed periodogram for α = 1.4; (c) Rectangular windowed periodogram for α = 1.1

c. **Hamming windowed periodogram for varying values of α:** Figure 1.10 represents the Hamming-windowed periodogram for two values of α. It is seen that for $\alpha = 10$, the width of the mainlobes are higher and there are more number of sidelobes, as compared to the rectangular windowed periodogram. Thus, this window does not fare well to differentiate the two different frequencies of sine waves. A higher threshold value of α (2.5) is needed to obtain two peaks. For a lower value of α (1.4), the resolving capabilities of the rectangular window are significantly better compared to the Hamming window.

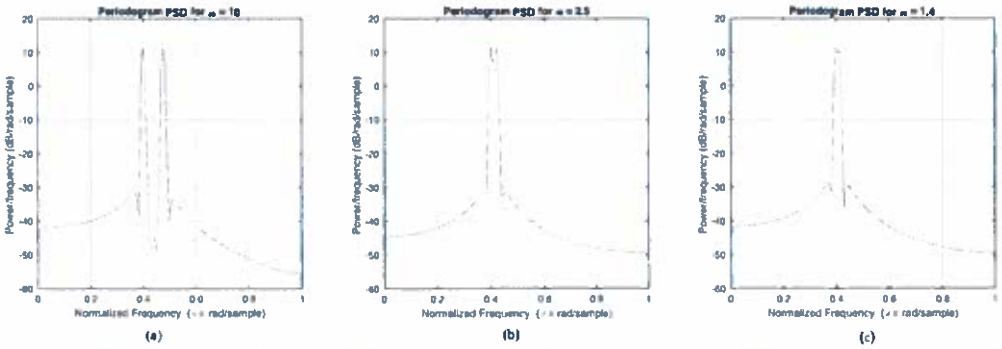


Figure 1.10 (a) Hamming windowed periodogram for α = 10; (b) Hamming windowed periodogram for α = 2.5; (c) Hamming windowed periodogram for α = 1.4

d. **Rectangular windowed periodogram to demonstrate spectral leakage:** Figure 1.11 represents the variation of a_2 with respect to constant value of a_1. The top row represents the spectrum estimates for $\alpha = 4$. It is seen that the amplitude threshold for good frequency resolution is very less. For an amplitude ratio of $a_2/a_1 = 10$, the frequencies cannot be distinguished. In Figure 1.13, the row at the bottom represents amplitude variations for $\alpha = 12$. It is seen in this case, that the frequencies are resolvable for larger amplitude ratios. This can be attributed to the larger spacing in the two frequencies caused by the larger α. Thus, there is a trade-off needed by frequency variation and amplitude variation to achieve good differentiation of the frequencies.

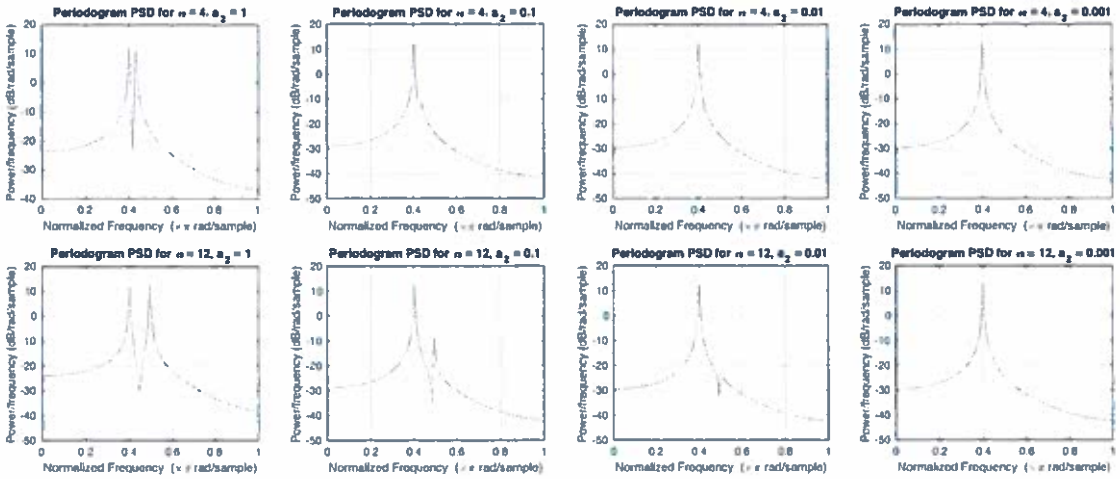


Figure 1.11 (Top row) Rectangular windowed periodogram for α = 4, and varying values of a_2 with respect to constant value of a_1; (Bottom row) Rectangular windowed periodogram for α = 12, and varying values of a_2 with respect to constant value of a_1

c. **Bartlett windowed periodogram to demonstrate spectral leakage:** Figure 1.12 represents the variation of a_2 with respect to constant value of a_1 for the Bartlett window. It is seen that there are extensive sidelobes, caused by the sidelobes of the Bartlett window. The top row represents the spectrum estimates for $\alpha = 4$. It is seen that the frequency estimates are better as compared to the rectangular window, albeit with many more side lobes. The bottom row, which represents the spectrum estimates for $\alpha = 12$, reinforces our conclusion from the previous subsection that the farther the frequency spacing, better the estimation. The estimation is better for larger ratios of a_2/a_1.

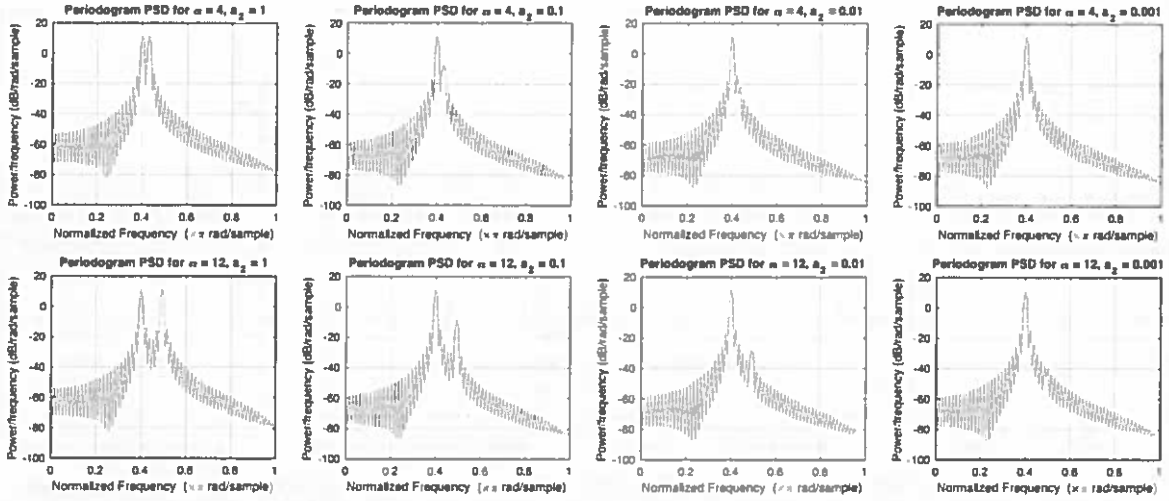


Figure 1.12 (Top row) Bartlett windowed periodogram for $\alpha = 4$, and varying values of a_2 with respect to constant value of a_1; (Bottom row) Bartlett windowed periodogram for $\alpha = 12$, and varying values of a_2 with respect to constant value of a_1

f. Chebyshev windowed periodogram vs Blackman-Tukey method: Figure 1.13 represents the variation of a_2 with respect to constant value of a_1 for a Chebyshev window. The top row represents the spectrum estimates for $\alpha = 4$ and the bottom for for $\alpha = 12$. In the case of $\alpha = 12$, it is seen that the Chebyshev window performs significantly better for larger values of the amplitude ratio. Hence, for the extreme case of a_1=1 and a_2=0.001, the periodogram produces good peaks, although the mainlobe peaks are not very sharp. It can be concluded that the constant sidelobe property helps in setting a higher amplitude ratio threshold while using a Chebyshev window.

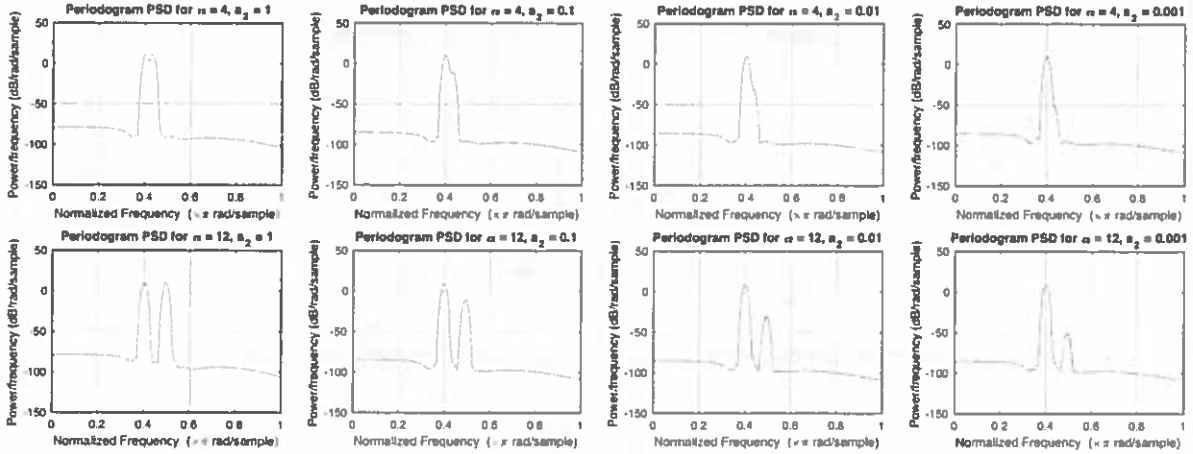


Figure 1.13 (Top row) Chebyshev windowed periodogram for $\alpha = 4$, and varying values of a_2 with respect to constant value of a_1; (Bottom row) Chebyshev windowed periodogram for $\alpha = 12$, and varying values of a_2 with respect to constant value of a_1

Figure 1.14 represents the variation of a_2 with respect to constant value of a_1 for the Blackman-Tukey method. It is seen that the peaks produced for the Blackman-Tukey method are sharper than the Chebyshev windowed periodogram, but overall its frequency distinguishing property is worse. The results produced in this case resemble the results obtained through the rectangular window, although the peaks are sharper.

Lack of consistencies for graphs.. some have grid some don't

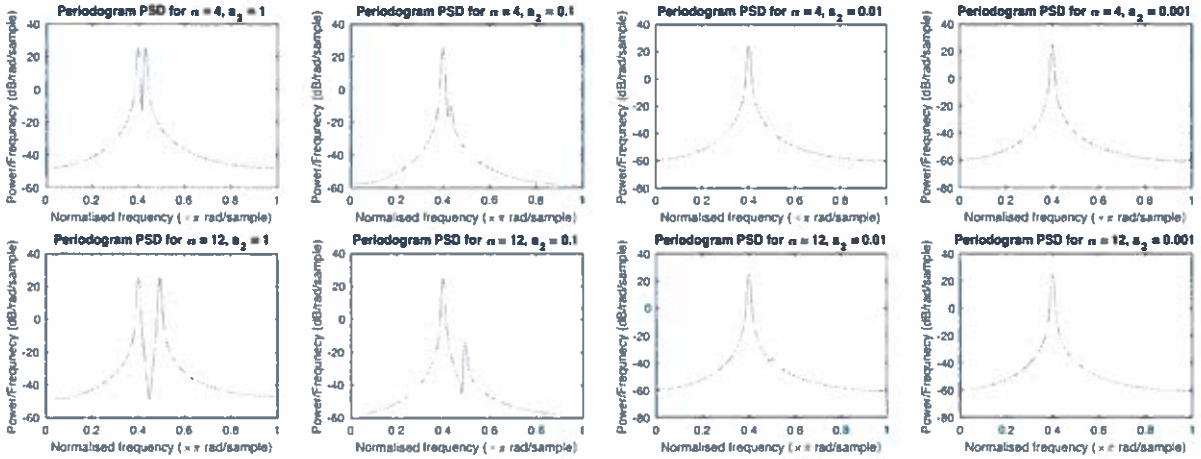


Figure 1.14 (Top row) Blackman-Tukey based periodogram for $\alpha = 4$, and varying values of a_2 with respect to constant value of a_1; (Bottom row) Blackman-Tukey based periodogram for $\alpha = 12$, and varying values of a_2 with respect to constant value of a_1

1.4. Periodogram-based Methods Applied to Real-World Data

a. The sunspot time series

The sunspot time series quantifies the number of sunspots in the surface of sun (known as the Wolf or Zurich number) against time. The variation in the Zurich number is measured over 300 years. Figure 1.15 (a) represents the variation of sunspot activity over 300 years. From the variations of maxima and minima, it is observed that sunspot activity repeats over a period of 11 years.

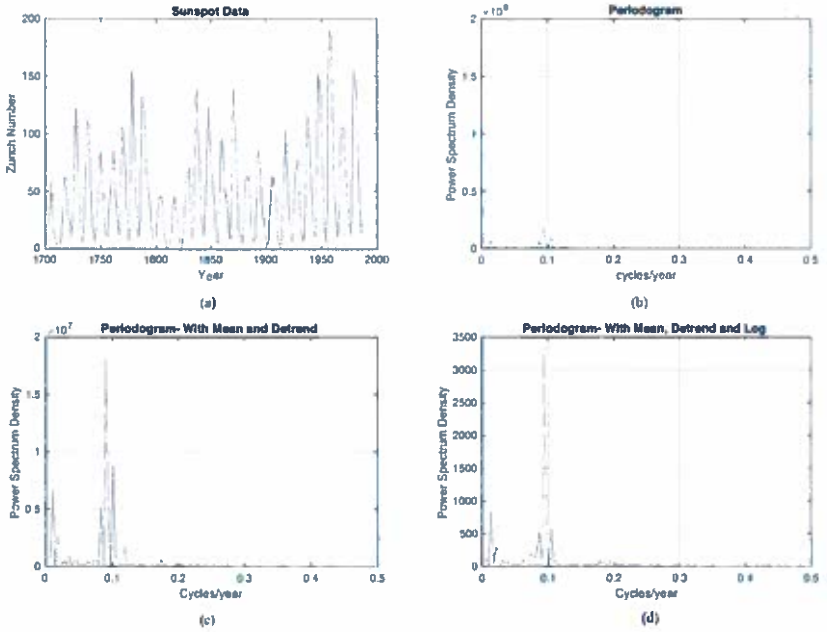


Figure 1.15 (a) Zurich number versus year (Sunspot activity) over 300 years; (b) Power Spectrum Density versus Frequency without any processing; (c) Periodogram obtained after removing mean and trend of the data; (c) Periodogram obtained after removing mean and trend of the data and applying log

The goal of the periodogram-based spectral estimation of sunspot data is to observe the variation in power of the time varied data over frequency cycles. By Parseval's theorem, the power spectrum is calculated as the squared version of the FFT of the signal. Figure 1.15 (b) represents the plot of the periodogram versus frequency. It is seen that there is a large value at the zeroth position. This value corresponds to the mean (or DC) component of the power spectrum. Other peaks are observed but are small in comparison to the mean value.

To have a concrete analysis, the mean value is removed for the spectrum. This presents a spectrum with comprehendible peaks as seen in Figure 1.15 (c). The largest peak occurs at a value of 0.09 cycles/year. In years/cycle, this corresponds to a value of 11 years. Hence, as expected, maximum activity of sunspot occurs approximately every 11 years. Removing the trend does not produce significant changes as the data varies in as a constant after a certain point of time.

Applying the log estimate, and removing its mean helps in creating a sharper difference in the peak values. The perception of important peaks improves, as the largest peak is enunciated, while the smaller peaks are subdued. The values are in a representable range varying between thousands, rather than a couple of millions. This is seen in Figure 1.15 (d).

b. Electroencephalogram (EEG) data

Analysis of the EEG data is achieved by obtaining the periodogram of the signal. The periodogram using a 10 second averaging window is represented in Figure 1.16. The peak corresponding to the alpha-rhythm is observed at 9.6 Hz and the interference is represented by a sharp peak at 50 Hz.

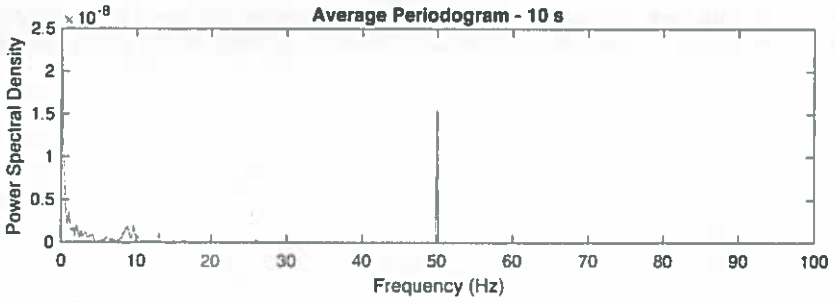


Figure 1.16 Periodogram obtained after using a 10 second averaging window. The periodogram is limited to the first 100 frequency (Hz) values, as this where the important information lies.

As the Steady State Visual Evoked Potential (SSVEP) is expected between 11 and 20 Hz, bandlimited periodograms are seen in Figure 1.17. It is seen from Figure 1.17 (a) – (c) that largest peak is obtained at 13 Hz. This corresponds to the SSVEP. The standard periodogram, as seen in Figure 1.17 (a) produces smaller peaks at other values, although the largest peak at 13 Hz is distinguishable. The averaged periodogram for 10 s and 5 s windows produces a smooth spectrum, where the largest peak is visible, while the sidelobes are relatively suppressed. The averaged periodogram for 1 s does not produce very good results, as seen in Figure 1.17 (d). The averaging tends to smooth the peaks significantly, reducing the sharpness needed to distinguish the largest peaks. Also averaging with this window length produces a large value at 11 Hz, which produces an incorrect analysis while determining the SSVEP. Hence it is concluded that averaging produces a smooth spectrum with correct peaks, provided the right window size is used.

good explanation and concluding remarks for the section

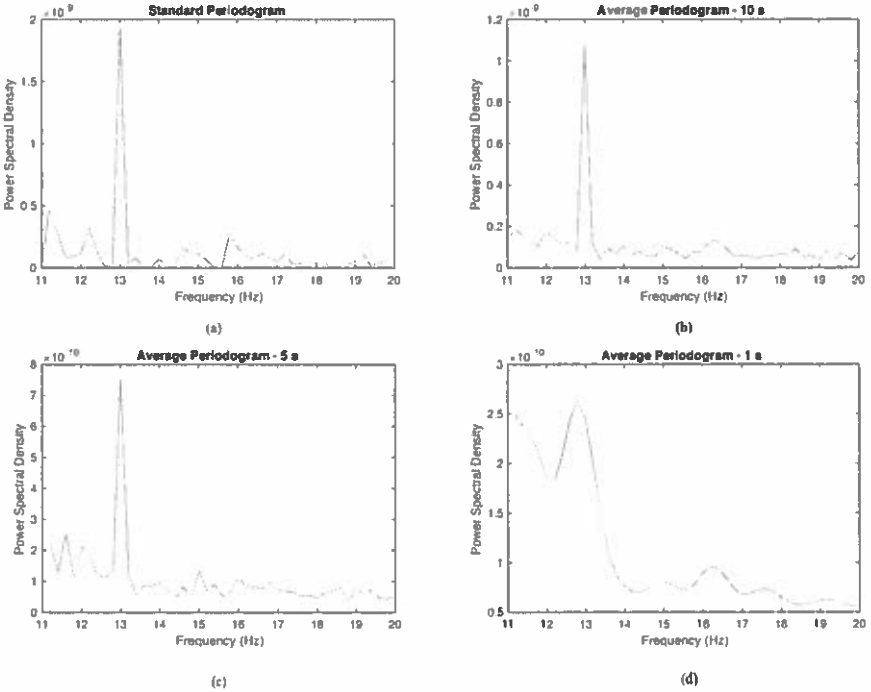


Figure 1.17 (a) Standard Periodogram of the EEG data; (b) Periodogram obtained after using a 10 second averaging window; (c) Periodogram obtained after using a 5 second averaging window; (d) Periodogram obtained after using a 1 second averaging window

2. Parametric and Line Spectra

2.1 Correlation Estimation

Biased and unbiased estimators

a. The biased and unbiased Autocorrelation functions (ACF) are calculated as per the equations:

$$\hat{r}(k) = \frac{1}{N}\sum_{n=k+1}^{N} x(n)x^*(n-k) \tag{7}$$

$$\hat{r}(k) = \frac{1}{N-k}\sum_{n=k+1}^{N} x(n)x^*(n-k), \quad 0 \le k \le N-1 \tag{8}$$

Equation (7) represents the biased ACF, while equation (8) represents the unbiased one. Initially, the ACF is calculated for a WGN signal. Its biased and unbiased ACF estimates are obtained as per the above equations and subsequently its correlogram is obtained. This is seen in Figure 2.1. It is observed that the biased ACF produces a large peak in the center, representing largest correlation and tapers as k moves towards N. The unbiased estimate has an increasing trend, becoming maximum as k moves towards N. The center peak is missing from the unbiased ACF. These are expected due to dividing scaling coefficient defining the unbiased ACF. The correlograms are seen in the Figure 2.1 as well. It is seen that the biased ACF estimate produces only positive values, while the unbiased estimated produce negative values. Figure 2.2 represents simulated results for a noisy sinusoidal signal and Figure 2.3 represents simulated results for filtered WGN signal.

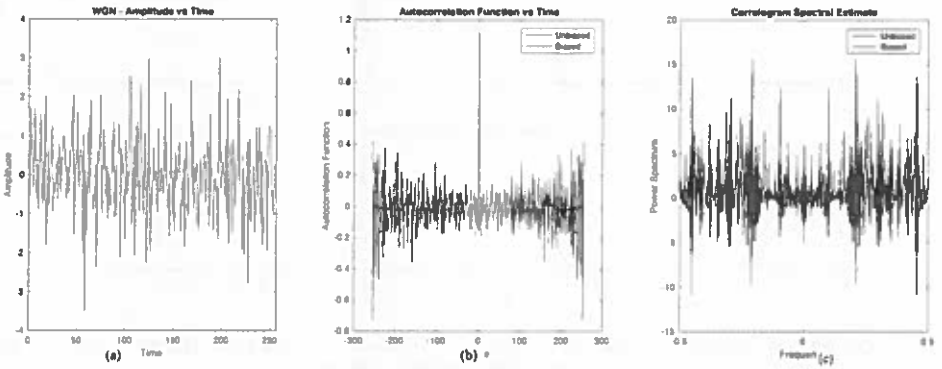


Figure 2.1 (a) White Gaussian Noise (WGN) signal; (b) ACF estimates; (c) Correlogram Spectral Estimates

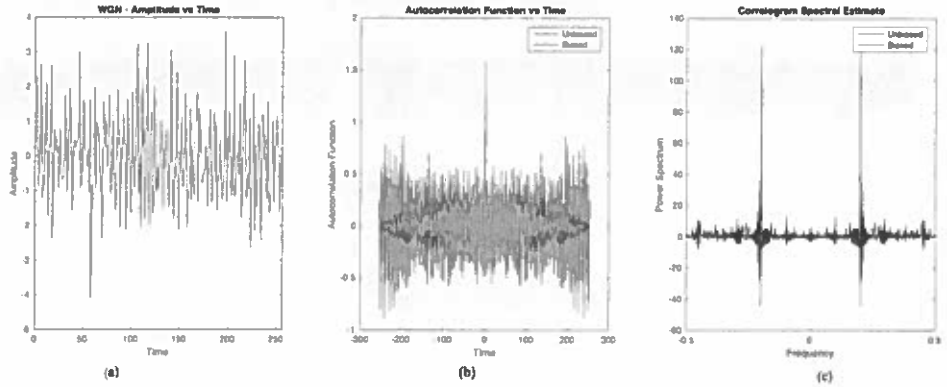
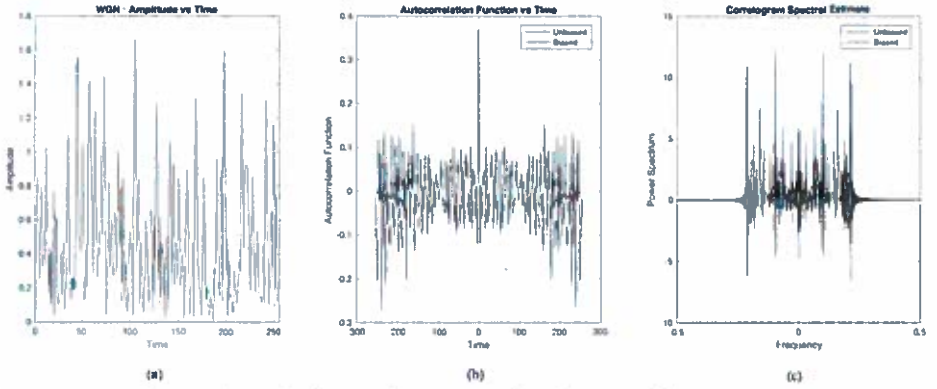


Figure 2.2 (a) White Gaussian Noise (WGN) signal; (b) ACF estimates; (c) Correlogram Spectral Estimates

Similar observations are made for the sinusoidal and filtered WGN signals, as seen for the WGN signal. Two sharp peaks are seen in the sinusoidal correlograms, as expected. It is observed from all three estimates that the unbiased estimates produce larger and more erratic peaks in the correlogram as k moves towards N.



As always, good explanations but figures could have been bigger.

Figure 2.3 (a) White Gaussian Noise (WGN) signal; (b) ACF estimates; (c) Correlogram Spectral Estimates

b. The PSD of 50 different estimates in the linear scale along with their mean is seen in Figure 2.4 (a). The standard deviation is seen in Figure 2.4 (b). It is seen that the difference between the mean value and the realizations are very similar. Hence, there is little to nor information that can be deciphered from this type of representation.

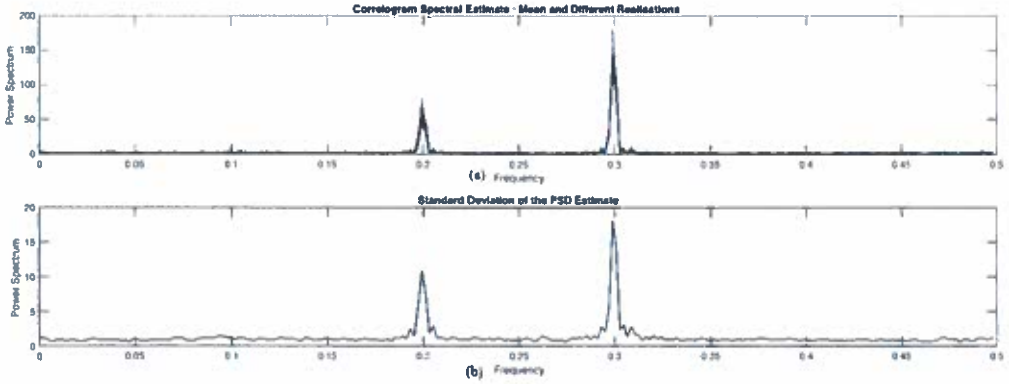


Figure 2.4 (a) Power Spectrum and Mean using Biased ACF Estimator. 50 realizations were used. The bold black line represents the mean value; (b) Standard deviation of PSD Values

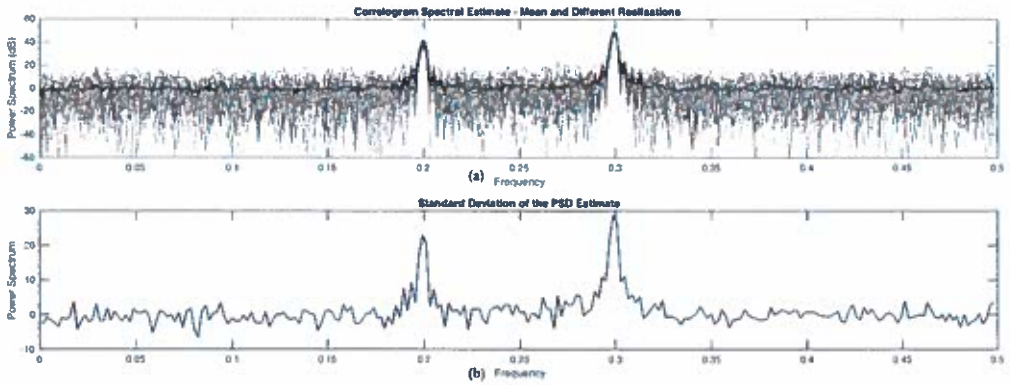


Figure 2.5 (a) Power Spectrum and Mean using Biased ACF Estimator using dB scale. The bold black line represents the mean value; (b) Standard deviation of PSD Values in dB

good explanation of why db scale is important

c. The estimated PSD of 50 different estimates in the dB scale along with their mean is seen in Figure 2.5 (a). The standard deviation is seen in Figure 2.5 (b). It is seen that the dB scale produces significant variation in the spectrum of different realizations. The mean produces good information relative to the differences in the estimates. The standard deviation too is more spread out, presenting newer information. The dB scale hence is produces a visual representation far superior to the linear scale.

d. The spectral estimates of complex exponential signals, consisting of two different frequencies are seen in Figure 2.6. The figure represents plots for different values of length. It is seen that increasing the length produces more identifiable peaks, but increases the number of sidelobes. Hence, there is a tradeoff between the sharpness of the peaks, and sidelobe power while deciding on the length of the sequence.

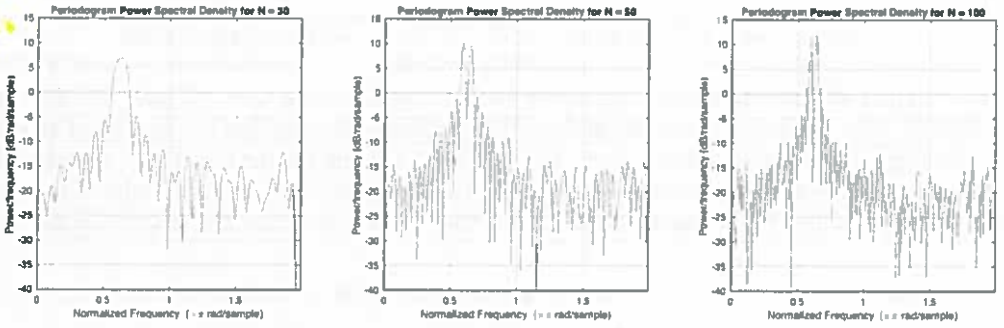


Figure 2.6 Power Spectrum for increasing values of N

Figure 2.7 represents the Power Spectrum estimates for a single f_1 value of 0.30, but varying f_2 values. A signal of length 30 is used in all the cases. It is seen that more the difference between the two frequencies, the better the periodogram is able to distinguish. This is expected as the window is able to distinguish farther frequencies better.

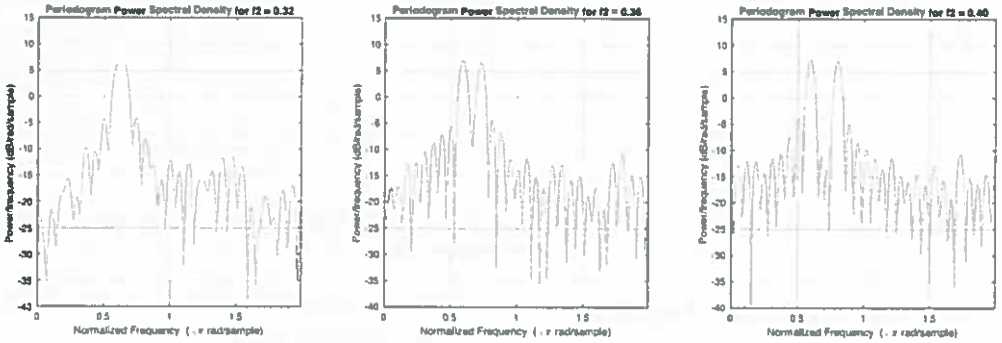


Figure 2.7 Power Spectrum for increasing values of f_2

e. The **MUSIC (Multiple Signal Classification)** algorithm is commonly used to estimate frequencies and detect the presence of a source sample in communication systems. In this context, it is used to estimate frequency by the generation of power spectra. To achieve this, the MUSIC algorithm uses the correlation matrix obtained from an input signal. The `corrmtx` command used to generate the data matrix to be used as the autocorrelation matrix for MUSIC estimation. It returns a Toeplitz matrix suitable for the command `pmusic`. This command generates the pseudo-spectrogram using the eigenspace method. The power spectrum plots obtained from the MUSIC algorithm are seen in Figure 2.8, for two different lengths of the input sequence. It is seen that the MUSIC algorithm is much more precise in detecting the peak values and the power of the side-lobes is significantly lesser or almost nill. This allows for easy differentiation of the peaks. It is also seen that increasing the length of the sequence affects the magnitude of the peaks. This can be attributed to incorrect magnitude scaling due to the autocorrelation matrix order. In order to achieve good peaks estimates, the right value of the sequence length must be used.

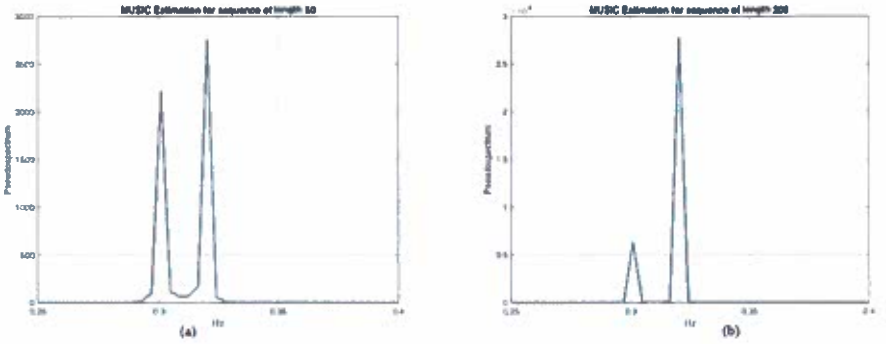


Figure 2.8 Power Spectrum for (a) sequence of length 50; (b) sequence of length 200

The mean and variance of different realizations of the MUSIC algorithm are seen in Figure 2.9. It is seen that the variance is obtained only in terms of the peaks and not the sidelobes, which makes it a very good algorithm for the estimation of frequencies. However, the variation in the standard deviation in the peaks is large, which may lead to different peak values in different realizations. This may lead to obtaining incorrect magnitudes of power spectrums. Hence, the MUSIC algorithm is very useful in determing the location of the peaks, rather than the magnitude of frequencies.

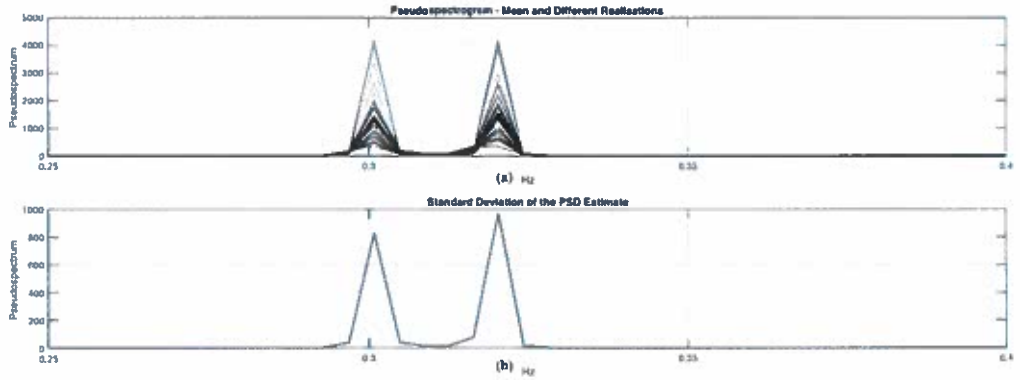


Figure 2.9 (a) Pseudospectrum and Mean for the MUSIC algorithms. The bold black line represents the mean value; (b) Standard deviation of Pseudospectrum values

2.2 Spectrum of Autoregressive Processes

good explanation of why the unbiased estimate can be prone to errors.

a. As seen in the previous section, the unbiased ACF estimates produce negative values which mean that they cannot be inverted for the calculation of Autoregressive (AR) parameters. Since methods of AR calculation, like the Yule Walker equation use the Levinson-Durbin Recursion algorithm, it is essential that the autocorrelation topelitz matrix used in any matrix equation is invertible. Hence, the unbiased ACF estimate may produce errors.

b. The Yule-Walker power spectral density estimates are produced, as seen in Figure 2.10. The Yule-Walker estimates of different orders are seen in the figure, along with a comparison of the true PSD. As expected, it is seen that all estimates below order 4 produce incorrect peaks. This is due to the fact that the ARMA input is a fourth order equation, relying on the past four estimates. Hence, a minimum order of 4 is required to estimate the peaks. It is seen in Figure 2.10 that the order 4 produces two peaks at the right location. Higher order power spectrums are also plotted to see a comparison. It is seen that increasing the order produces close estimates to the true PSD value. However, after a certain order (10 in this case), the peaks are not rounded but possess rough edges. This may presen problems while using a differentiating algorithm to estimate the peaks. Hence, a good high order estimate like 8 or 10 can be used, while still obtaining good peaks and having reasonable computation complexity.

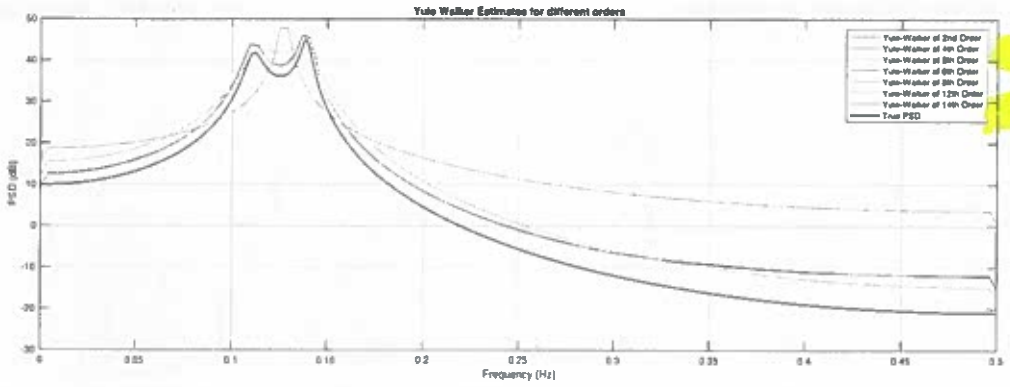


Figure 2.10 (a) Yule Walker Power Spectral Density Estimates for various orders compared with True PSD (bold black plot).

c. The above experiment is repeated for 10,000 samples and the results are consolidated in Figure 2.11. It is seen that under modelling does not produce the right peaks. Whereas, over modeling produces block-like variations of the peak, rather than a smooth peak. This can be attributed to the fact that the model is not 'over-fitted' to the given data points, which provides less flexibility. Hence the right order to be used for large samples is the model value itself, i.e., 4.

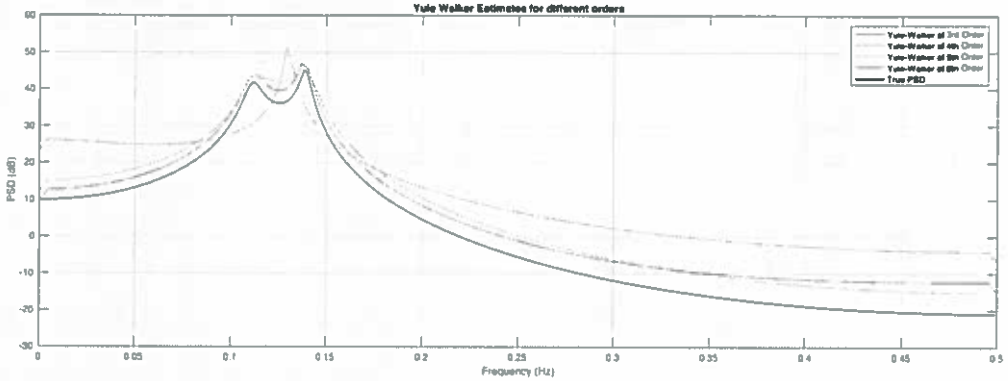


Figure 2.11 (a) Yule Walker Power Spectral Density Estimates for various orders compared with True PSD (bold black plot). The above plot is obtained for an input of 10,000 samples.

2.3 Principal Component Analysis

a. The SVD of X and X_{noise} are seen in Figure 2.12 (a). It is seen that X possesses 3 large values, while all other components are 0. This indicates that its rank is 3. In comparison, X_{noise} possesses 3 large values and 7 relatively small values. Hence, it also possesses a rank of 3, albeit it is harder to distinguish. Thus, it is seen that noise increases the values of all the singular components, more specifically those components that were not so significant in the noiseless case. Figure 2.12 (b) represents the squared error between the X and X_{noise}. It is seen that noise affects non-essential dimensions more than the essential ones. When noise is increased to a level such that it is comparable to the signal power, the SVD decomposition becomes indistinguishable. This is seen in Figure 2.11 (c). Here, it is seen that there is no clear large value. All singular values seem to be similar, making their ranks indistinguishable.

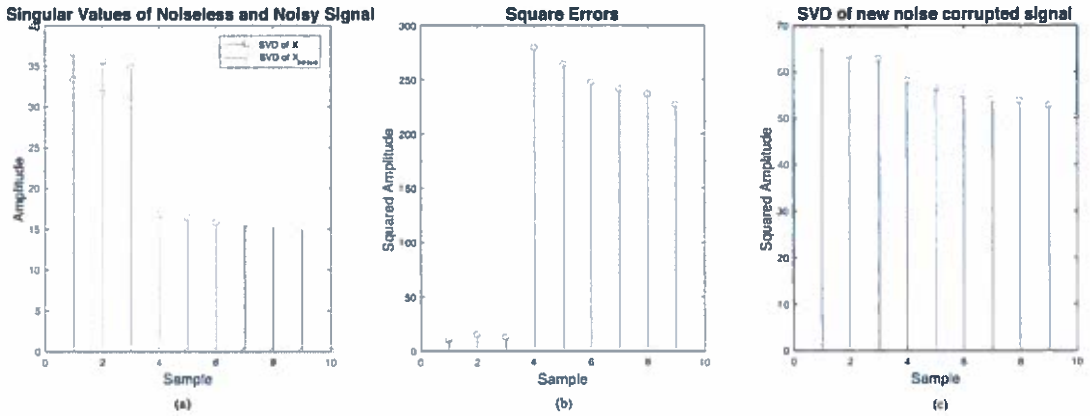


Figure 2.12 (a) SVD of X and X_{noise} ; (b) Squared Errors between X and X_{noise} ; (c) SVD of noise corrupted signal having 3.5X noise compared to X_{noise}

b. Only the first 3 principal components were taken from X_{noise} and Singular Value Reconstruction was performed. Figure 2.13 represents the errors obtained on an average for all 1000 training samples, for the 10 features. It is seen that taking only the principal components significantly reduces the error across all the 10 features. This is attributed to the fact that the unnecessary singular values are completely eliminated, thereby reducing the effect of noise contribution.

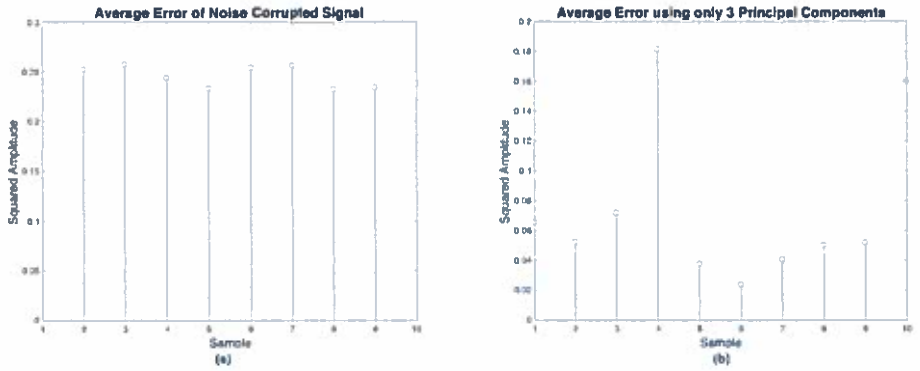


Figure 2.13 (a) Average Error of Noise Corrupted Signal; (b) Average Error using only 3 Principal Components of the SVD Decomposition

c. The Ordinary Least Squares (OLS) and the Principal Component Regression (PCR) methods are used to obtain difference estimates of the regression matrix [4]. From these matrices, an estimate output is calculated and compared with the original output. The errors are for each feature of the output is seen in Figure 2.14. It is seen that the OLS estimate produces slightly smaller errors as compared to the PCR, while using the training set to estimate the data. While using the testing set, it is seen that the PCR produces smaller errors. This can be attributed to the fact that the OLS regression matrix is more overfitted to the training set, producing better results for training samples; whereas the PCR method is well trained to be able to distinguish any data which is outside the training set.

interesting comparison beween OCR and PCR

d. Table 2.1 represents the mean square errors obtained by using an ensemble of test data to test the regression coefficients. It is seen that the PCR estimate produces lesser error as compared to the OLS estimate. Thus the PCR is better for use in machine learning as compared to the OLS. In machine learning, the testing data is often much different compared to the training data. Hence the model chosen should be general enough so that it can accommodate all variants of data from a class, while not being overfitted. As the PCR relies on the principal components of the SVD, while the OLS relies on the entire autocorrelation matrix, the effectiveness of PCR for any test data is expected

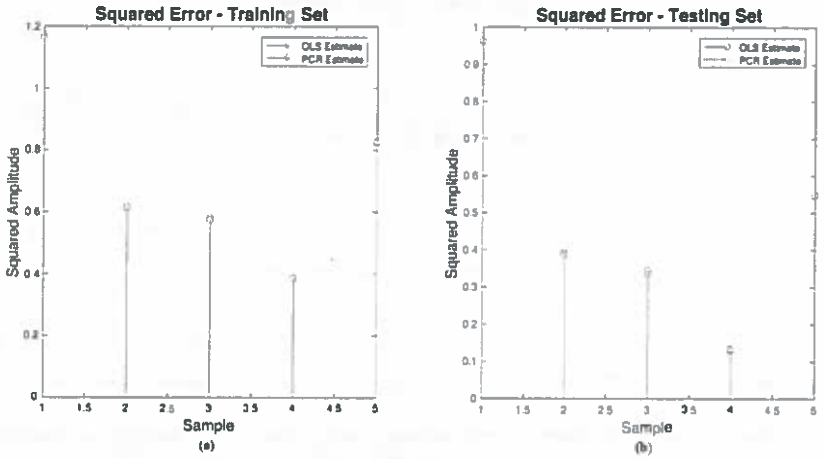


Figure 2.14 (a) Squared Error using Training Set for estimation; (b) Squared Error using Testing Set for estimation

Table 2.1 Squared Error for the OLS and PCR Estimation

	Squared Error
OLS	0.459
PCR	0.4532

2.4 Real World Signals: Respiratory Sinus Arrhythmia from RR-Intervals

a. The plots obtained from the three trials, i.e., regular, fast and slow breathing are represented in Figure 2.15, 2.16 and 2.17 respectively. The plots are obtained with respect to seconds per breath, i.e, inverse of frequency. This is done for ease of understanding and calculation of results described in the next section. The averaged periodograms were obtained for window sizes of 100s and 200s.

b. From the periodograms, the maximum peaks can be seen for all three cases (standard, 100s average and 200s average). The difference lies in the sharpness of the peaks along with the power of the side lobes. In the case of the standard periodogram, the sharpest peaks are seen with large sidelobe values, whereas the 200s average periodogram does not possess very sharp peaks and gradual, smooth increase in peak values. The peaks observed in each case are summarized in Table 2.2. The peaks in sec/breath are inverted to obtain the peaks in breath/second and are subsequently multiplied by 60 to obtain the frequency/minute. On an average, it is seen that the normal breathing rate is 20 breaths/min, for fast breathing, it is 25 breaths/min and for slow breathing, it is 7.5 breaths/min. The normal breathing rate corresponds to the theoretical breathing rate of 20 breaths/second for a human adult.

Table 2.2: Peaks observed from the periodograms and their corresponding frequency of breathing

	Regular		Fast		Slow	
	Peak (Sec/breath)	Frequency	Peak (Sec/breath)	Frequency	Peak (Sec/breath)	Frequency
Standard Periodogram	2.93	20.4778157	2.408	24.91694352	7.967	7.531065646
100s Avg Periodogram	2.936	20.4359673	2.39	25.10460251	7.983	7.515971439
200s Avg Periodogram	2.922	20.5338809	2.4	25	8.016	7.48502994
Average		20.48255464		25.00718201		7.510689008

good use of tables. were not used previously however, and could have gained some extra marks.

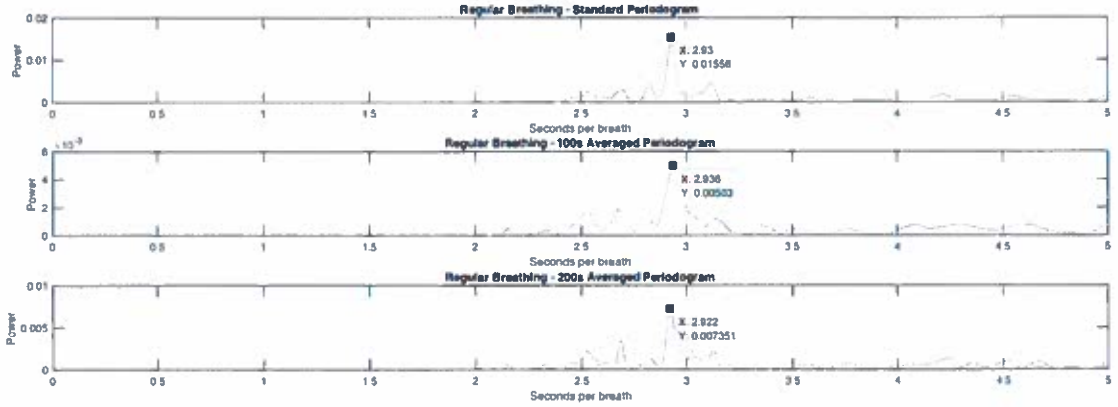


Figure 2.15 Standard periodogram and averaged periodograms obtained for the case of regular breathing. The peaks and their locations have been highlighted.

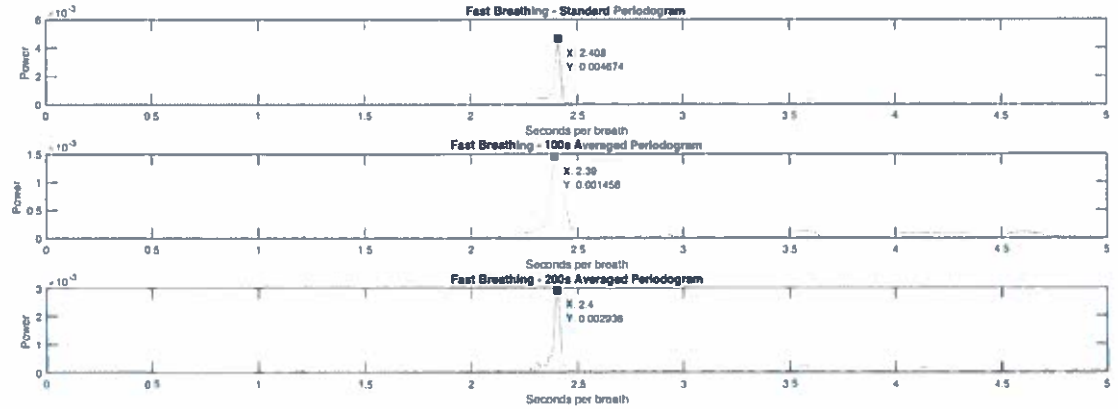


Figure 2.16 Standard periodogram and averaged periodograms obtained for the case of fast breathing. The peaks and their locations have been highlighted.

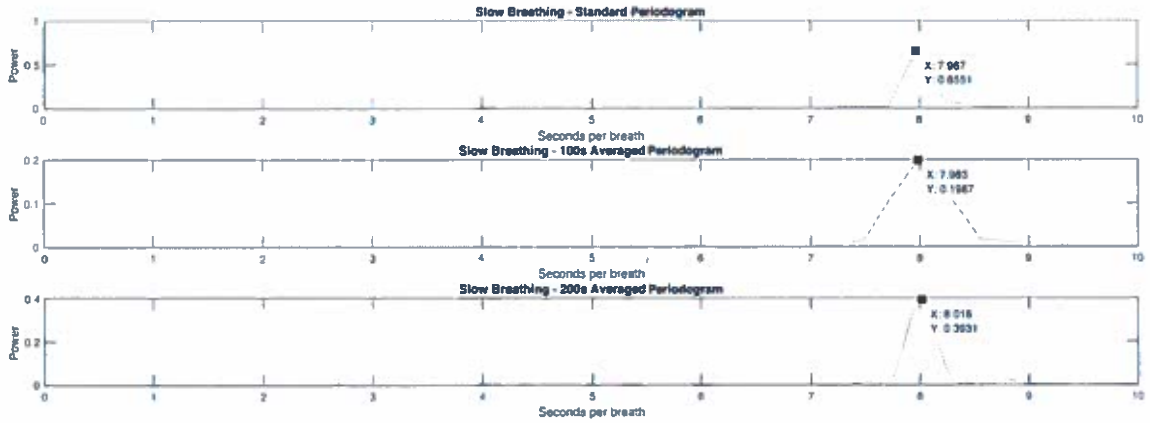


Figure 2.17 Standard periodogram and averaged periodograms obtained for the case of slow breathing. The peaks and their locations have been highlighted.

c. Figure 2.18 represents the AR spectrum obtained for the RRI signals. An order of 14 is used here. It is observed that unlike the spectrogram plots, the AR spectrum does not possess any side lobes and the peaks are clearly visible in the plots. However, the plots may be too smooth so distinguishing the right point on the peak may be difficult. In Figure 2.18, the power is plotted against second/breath (frequency) to present another variation in representation as opposed to breaths/second seen in the previous section. The peaks and the rate of respiration per minute are seen in Table 2.3. The obtained frequencies are similar to those obtained using the periodogram.

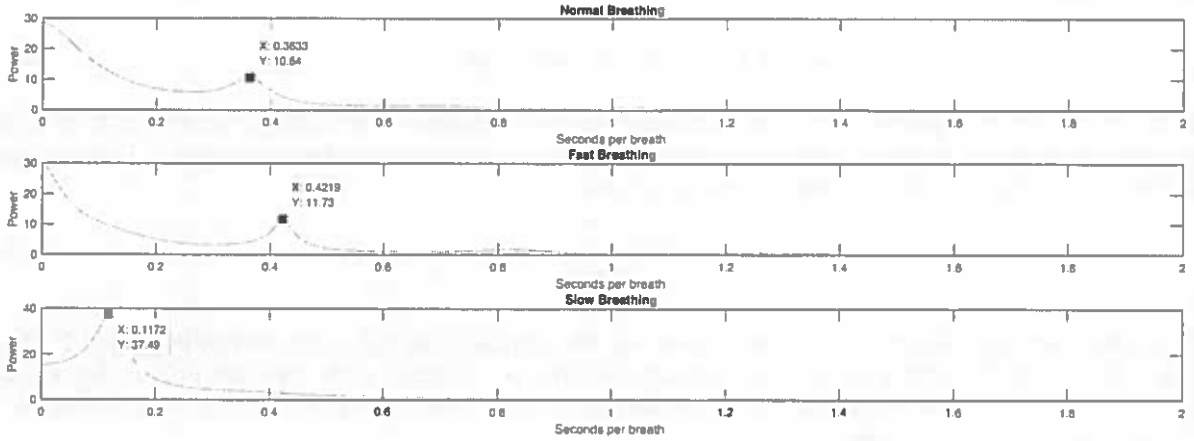


Figure 2.18 AR spectrum obtained for the case of regular, fast and slow breathing. The peaks and their locations have been highlighted.

Table 2.3: Peaks observed from the AR Spectrums and their corresponding frequency of breathing

	Regular		Fast		Slow	
	Peak (Breath/Sec)	Frequency	Peak (Breath/Sec)	Frequency	Peak (Breath/Sec)	Frequency
AR Spectrum	0.3633	21.798	0.4219	25.314	0.1172	7.032

3. Adaptive Signal Processing

3.1 The Least Mean Square (LMS) Algorithm

The Least Mean Square algorithm targets to minimize the instantaneous square error $J(n) = 0.5e^2(n)$ where $e(n) = x(n) - w^T(n)x(n)$. $w(n)$ is the coefficient vector of the filter having length L and $x(n)$ is the data vector. $x(n)$ is of length L and is taken N-L times from the complete data vector. The weight vector is updated according to the equation:

$$w(n+1) = w(n) + \mu x(n)e(n) \tag{9}$$

Where μ is the adaption gain and $x(n)e(n)$ is the instantaneous estimate of the negative gradient of J, given by $\hat{x}(n)$ For a given input, the instantaneous correlation matrix is calculated from an input of length L. From this, the maximum eigenvalue is used to determine the step size μ as:

$$0 < \mu < \frac{2}{\lambda_{max}} \tag{10}$$

a. **Step Size**: The AR(2) function described is generated. The correlation matrix is obtained for every past 2 values of the instantaneous estimate. Hence, it is assumed that the first two elements of the input are 1. Once this is done, the `xcorr` function is used to generate the correlation matrix. Since MATLAB generates symmetric values, the 2 X 2 matrix can be generated using:

```
r = xcorr(x(i-2:i-1));
R = [r(2), r(3); r(3), r(2)];
```

Theoretically, the autocorrelation matrix is of the form:

$$R = \begin{bmatrix} E\{x^2(n-1)\} & E\{x(n-1)x(n-2)\} \\ E\{x(n-1)x(n-2)\} & E\{x^2(n-1)\} \end{bmatrix} \tag{11}$$

Theoretical derivation is missing

By solving this equation, with respect to the AR(2) process given in question, the theoretical correlation matrix is:

$$R = \begin{bmatrix} 25/27 & 25/54 \\ 25/54 & 25/27 \end{bmatrix}$$

In the simulation, the average of the largest eigenvalue λ_{max} is calculated over the entire range of values. This is used to define the range of μ values at which the LMS algorithm would converge. From experimentation, it is seen that λ_{max} is 1.127, which implies that μ can take the value 1.774. From the theoretical value, we know that λ_{max} is 1.389, which implies that μ has to be less than 1.44.

b. **Mean Squared Error:** The LMS predictor is obtained for the AR(2) signal and its mean square error is plotted in Figure 3.1 for two different step sizes. It is seen that there is no clear convergence that is seen for the LMS. Subsequently, the same is performed for 100 instances and is averaged. This is seen in Figure 3.2. It is seen that, for both the step sizes the error converges indicating that the weights have obtained roughly constant values. It is also seen that for $\mu = 0.05$, the error converges faster. This is seen in the steeper drop as compared to for $\mu = 0.01$. Thus, an appropriate step size is needed to ensure proper convergence.

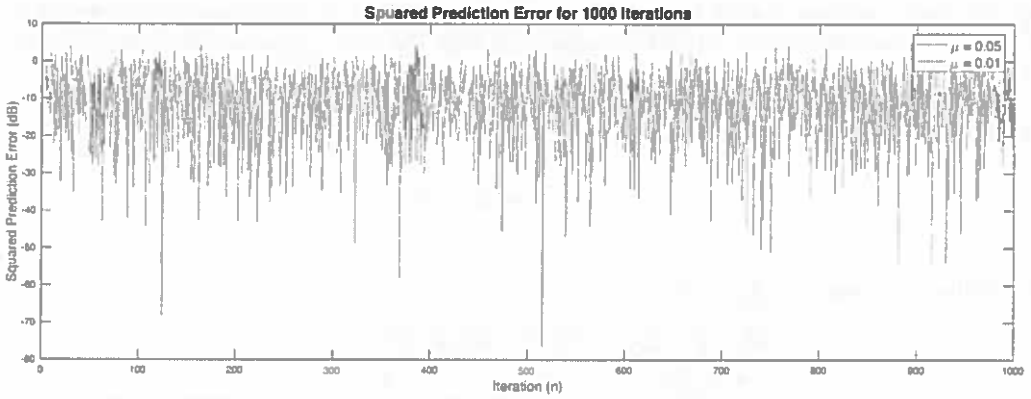


Figure 3.1 Squared Prediction Error for Single Instance

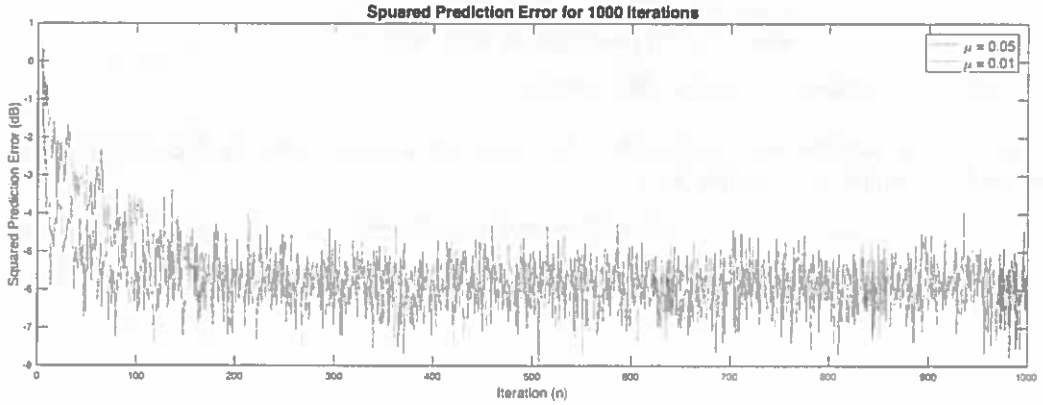


Figure 3.2 Averaged Squared Prediction Error for 100 Instances

c. Miadjustment: The trace of the correlation matrix is calculated using the theoretical correlation matrix seen in the previous two sections. Thus, $\boldsymbol{Tr\{R\} = 1.8519}$. Using the definitions described in [5], the theoretical and practical values of the misadjustment are:

Table 3.1: Theoretical Misadjustment values versus Practical Misadjustment for two trials.

Step Size	Theoretical	Practical (Trial 1)	Practical (Trial 2)
0.05	0.0453	0.0413	0.0492
0.01	0.0093	0.0146	0.0104

Two cases of the practical results are seen, each for 100 instances of the LMS implementation. It is seen that the practical values of the Misadjustment lie close to the theoretical values.

d. **Steady State Values**: The steady state values are obtained by averaging the last 400 values obtained using the LMS algorithm. These are shown in Table 3.2

Table 3.2: True coefficients of the AR(2) system versus Steady State Values for two step sizes: 0.05 and 0.01

True Coefficient	Obtained - 0.05	Obtained - 0.01
0.1	0.1027	0.1035
0.8	0.6939	0.7447

Even though the larger step size of 0.05 produces better steady state for one coefficient, it is seen that the smaller step size of 0.01 produces the best steady state result when both the coefficients are considered. Its steady state values are closer to the actual values.

e. For **Leaky LMS**, the cost function is defined as:

$$J_2(n) = \frac{1}{2}(e^2(n) + \gamma\|w(n)\|_2^2) \tag{12}$$

The gradient of the cost function is given by:

$$\nabla J_2(n) = \frac{1}{2}(\nabla e^2(n) + \gamma\nabla\|w(n)\|_2^2)$$
$$\Rightarrow \nabla J_2(n) = -e(n)x(n) + \gamma w(n) \tag{13}$$

Substituting this in Equation (9):

$$w(n+1) = w(n) + \mu e(n)x(n) - \mu\gamma w(n)$$
$$w(n+1) = (1 - \mu\gamma)w(n) + \mu e(n)x(n) \tag{14}$$

Thus, the weight update function for Leaky LMS is defined.

f. The **Leaky LMS** is implemented for the AR(2) function. The squared errors are seen in Figure 3.3. Similar observations hold as compared to Section 3.1.a.

good section overall, with appropriate use of diagrams and tables.

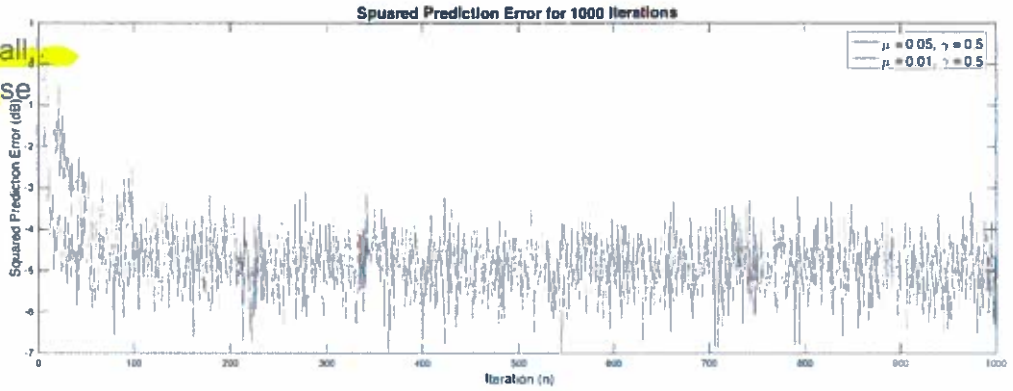


Figure 3.3 Averaged Squared Prediction Error for 100 Instances of Leaky LMS

The steady state values for different values of leakage coefficients, γ and step sizes, μ are seen in Table 3.3.

Table 3.3: Steady State Values obtained for various values of leakage coefficients (γ) and step size (μ)

Leakage Coefficient	0.1		0.5		0.8	
Step Size	a_1	a_2	a_1	a_2	a_1	a_2
0.05	0.11	0.63	0.09	0.35	0.11	0.31
0.01	0.15	0.66	0.12	0.52	0.13	0.38

It is seen that as the leakage coefficients (γ) increases, the steady state values of the LMS approximation move away from the original coefficient values. This can be attributed to the fact that the leakage coefficient introduces new values in the correlation matrix of the input, that helps the system converge. By increasing the value of γ, additional noise is being introduced into the system which disrupts the approximation done by the LMS. Hence,

a small value of γ can be used for the cases when the LMS does not converge, but introducing a high value increases the noise, which causes error in detection.

3.2 Adaptive Step Sizes

Adaptive Step Sizes are used to increase the convergence rate, while keeping a good steady state error. To achieve this, the step size variation is also made an instantaneous optimization problem. The learning rate $\mu(n)$ is now varied according to the Gradient Adaptive Step Size Algorithm (GASS). The learning rate updating is given by:

$$\mu(n+1) = \mu(n) + \rho e(n)x^T(n)\psi(n) \tag{15}$$

where $\psi(n)$ can have the following forms based on the algorithm:

$$\begin{gathered}\text{Benveniste: } \psi(n) = [I - \mu(n-1)x(n-1)x^T(n-1)]\psi(n-1) + e(n-1)x(n-1) \\ \text{Ang \& Farhang: } \psi(n) = \alpha\psi(n-1) + e(n-1)x(n-1), 0 < \alpha < 1 \\ \text{Matthews \& Xie: } \psi(n) = e(n-1)x(n-1)\end{gathered} \tag{16}$$

a. **GASS Algorithms**: The algorithms described above are implemented for an MA(1) system defined in [6]. The weights along with their errors are seen in Figure 3.4. These are also compared with the standard LMS algorithm.

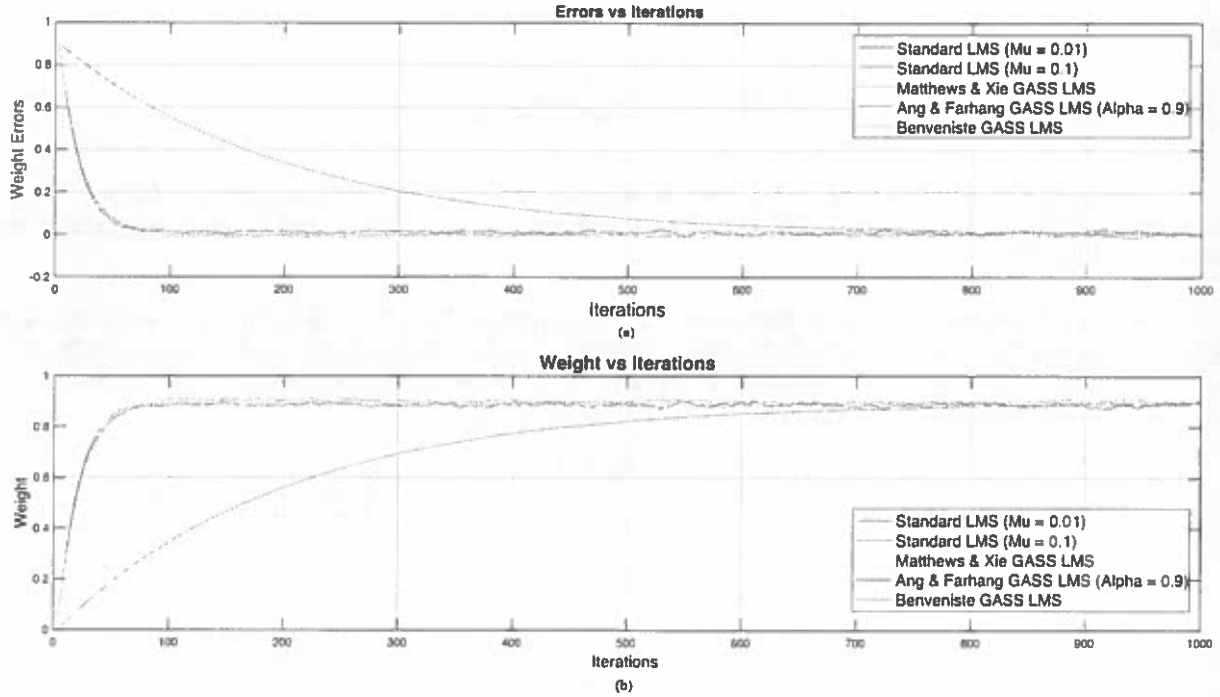


Figure 3.4 (a) Weight error curves for different GASS algorithms versus standard LMS; (b) Weight convergence curves

It is seen that all algorithms converge to weight values close to 0.9. This corresponds to the definition of the MA(1) system. The standard LMS algorithm with less μ value (0.01), possesses the smoothest convergence. However, its convergence speed is extremely low. The LMS with high μ (0.1), along with all the GAS algorithms perform significantly better in terms of convergence speed. But they are not very smooth, although the convergence to 0.9 is achieved. In this regard, the Benveniste algorithm produces the best trade off in terms of convergence speed and steady state values.

some sporadic grammatical mistakes

b. **Normalised LMS**: The Normalised LMS is used to modify the weight updating equation, so that the updation is proportional to the input. By varying the value of the adaption gain by putting it out of the user's control and making it dependant on the input, the LMS algorithm become more robust to real time signals. This is the basis of the Normalised Least Mean Square Algorithm. To derive the LMS condition, the weight updation from Equation (9) is rewritten as:

$$w(n+1) = w(n) + \mu x(n) e_p(n) \tag{17}$$

where $e_p(n)$ is the posteriori error defined as $e_p(n) = d(n) - x^T(n)w(n+1)$. Substituting Equation (17) in this definition:

$$\begin{aligned} e_p(n) &= d(n) - x^T(n)w(n) - \mu x^T(n)e_p(n)x(n) \\ \Rightarrow [1 + \mu x^T(n)x(n)]e_p(n) &= d(n) - x^T(n)w(n) \\ \Rightarrow e_p(n) &= \frac{d(n) - x^T(n)w(n)}{[1 + \mu x^T(n)x(n)]} \end{aligned} \tag{18}$$

It is known that $e(n) = d(n) - x^T(n)w(n)$ from the definition of LMS. Substituting $e_p(n)$ in Equation (17):

$$w(n+1) = w(n) + \mu x(n) \frac{e(n)}{[1 + \mu x^T(n)x(n)]} \tag{19}$$

Rearranging:

no distinction between vector and scalar quantities

$$w(n+1) = w(n) + \frac{1}{[1/\mu + \mu x^T(n)x(n)]} x(n)e(n) \tag{20}$$

This is similar to the definition given in [6], with the adaption constant $\beta = 1$ and regularization factor $\epsilon = 1/\mu$. Thus the conditions of Normalised LMS have been derived, and it is seen that the updation is proportional to the input at every instant.

c. **Generalized Normalized Gradient Descent (GNGD) Algorithm**: The GNGD algorithm was implemented as defined in [6] and the outputs are seen in Figure 3.5. It is seen that the GNGD algorithm converges much fsater as compared to the Benveniste GASS algorithm. It converges at 0.9, however it possesses more random oscillations around the steady state value as compared to the Benveniste algorithm.

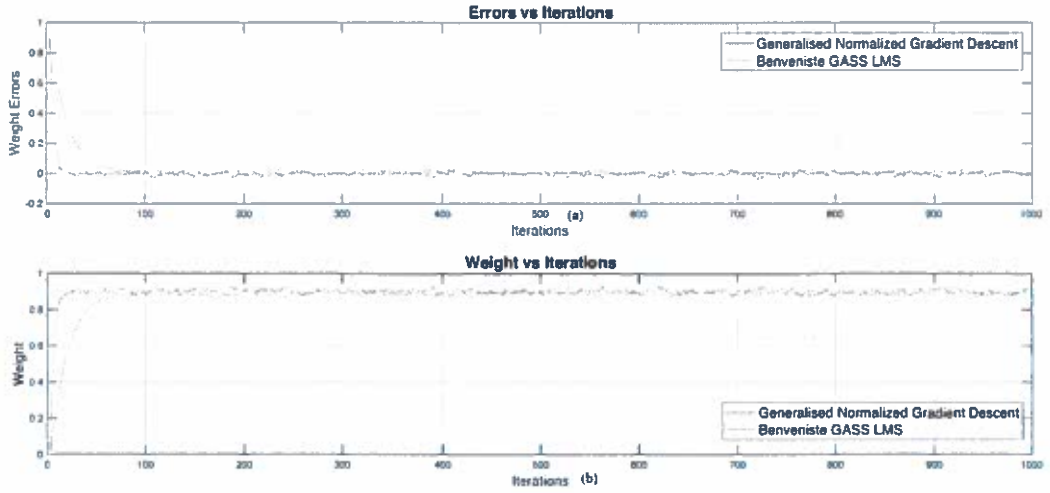


Figure 3.5 (a) Weight error curves for the GNGD algorithm versus standard Benveniste GASS LMS; (b) Weight convergence curves

It is observed that the computation power required for the GNGD estimation is significantly lower as compared to the Benveniste GASS algorithm. The Benveniste algorithm requires around more multiplications and additions, as compared to the GNGD algorithm. This makes the GNGD algorithm highly superior in terms of computation and higher convergence rate.

3.3 Adaptive Noise Cancellation

a. **Adaptive Line Enhancer**: The Adaptive Line Enhancer (ALE) was implemented in MATLAB using the LMS algorithm. Figure 3.6 (a) showcases the noise corrupted signal with reference to the original signal, whereas Figure 3.6 (b) represents the signal obtained after applying the ALE algorithm. It is seen that the noise is significantly reduced and the obtained signal almost follows the original signal with slight deviations. The step size used was 0.001 and the filter order used was 20. To determine the value of the minimum delay Δ, the Mean Square Error of the noise filtered signal was obtained with respect to the original signal, from different values of filter order and plotted against delay values. This is shown in Figure 3.7. It is seen that the least mean square error is obtained for a delay value of 2 for all filter orders. Hence, Δ = 2 is seen to produce good results. mathematical proof is missing

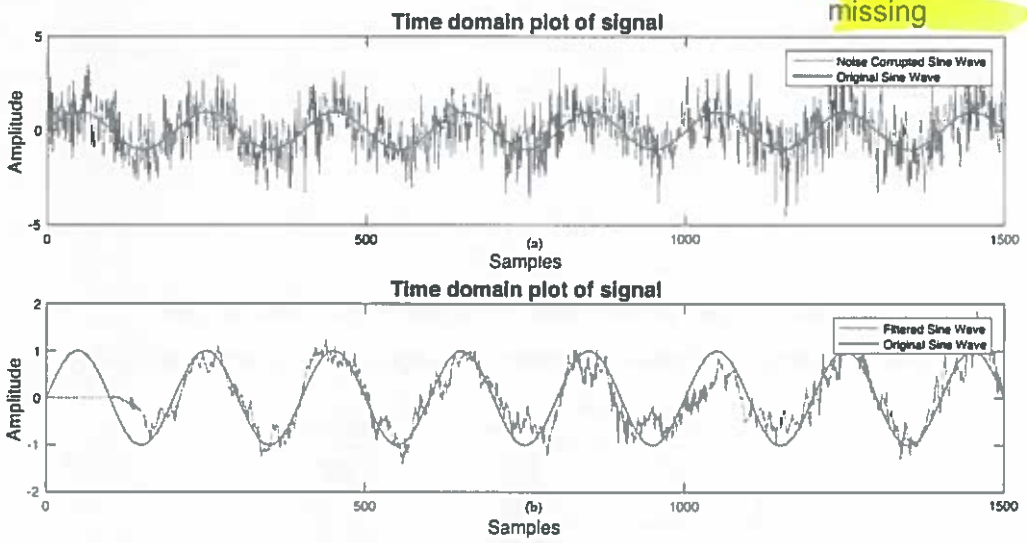


Figure 3.6 (a) Time domain plots of noise corrupted signal versus original signal; (b) Time domain plot of filtered sine wave vs original signal.

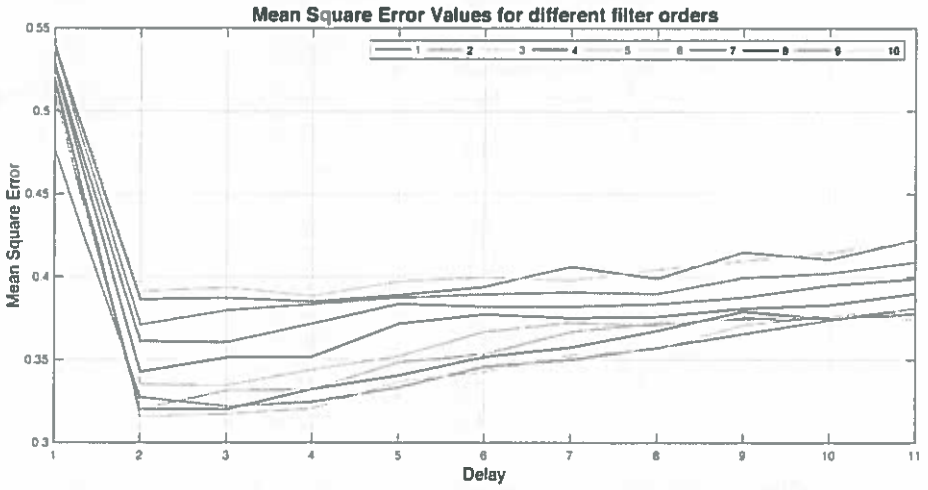


Figure 3.7 Mean Square Errors plots against different orders for the determination of least delay

b. It is seen that both delay Δ and order M play an important role in Denoising using the ALE algorithm. Figure 3.8 shows the effect of increasing the order and delay values in the mean square error of the denoised signal. It is seen that the best delay value is at $\Delta = 2$, as seen earlier. The MSE is dependent on the filter order as the correct set of continuous samples would help in minimizing the errors in a given case. It is also seen that the minimum errors occur for order 10 and 15. As computation increases with increasing order, it is essential to choose an order value between 5 and 15 as this would produce both the minimum errors and least computation time. Table 3.4 summarizes the MSE and computation time for different order values between 5 and 15. It is seen here that the best performance is obtained for $M = 6$. Hence, using an order of 6 would be computationally effective and produce good MSE.

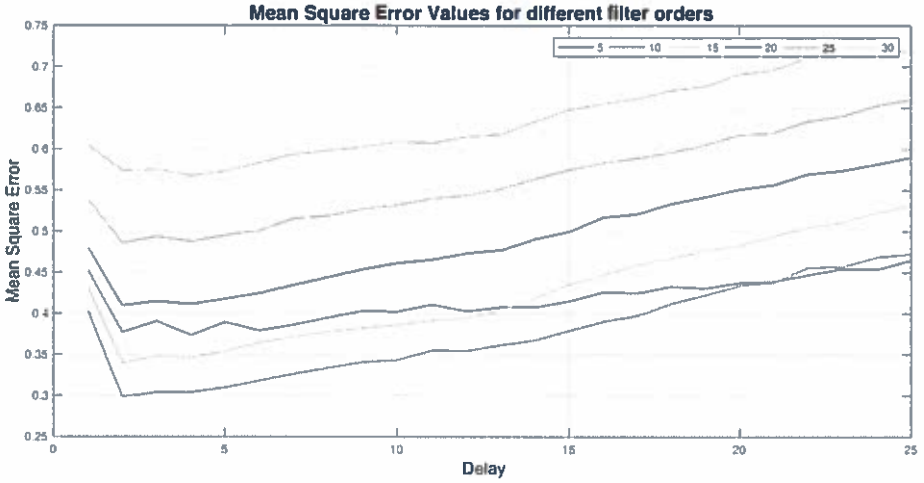


Figure 3.8 Mean Square Errors plots against different orders and delay values

Table 3.4: Mean Square Prediction Errors (MSPE) and computation time for different filter orders

Order	MSPE	Time (secs)
1	0.4	0.0091
2	0.33	0.0186
3	0.32	0.0166
4	0.32	0.0170
5	0.31	0.0201
6	0.31	0.0142
7	0.32	0.0223
8	0.32	0.0188
9	0.33	0.0199
10	0.34	0.0174
11	0.35	0.0150
12	0.36	0.0166
13	0.38	0.0163
14	0.39	0.0152
15	0.4	0.0141

good use of tables and graphs

c. **Adaptive Noise Cancellation**: The Adaption Noise Cancellation (ANC) algorithm is implemented and its results are reported in Figure 3.9. It is seen that the ANC algorithm becomes completely noiseless over increasing amount

of iterations. The difference in Mean Square Errors between the ALE and ANC algorithms is seen in Table 3.5. It is seen that the ANC performs much better as compared to the ALE, which is also observed intuitively by observing Figure 3.6 and 3.9

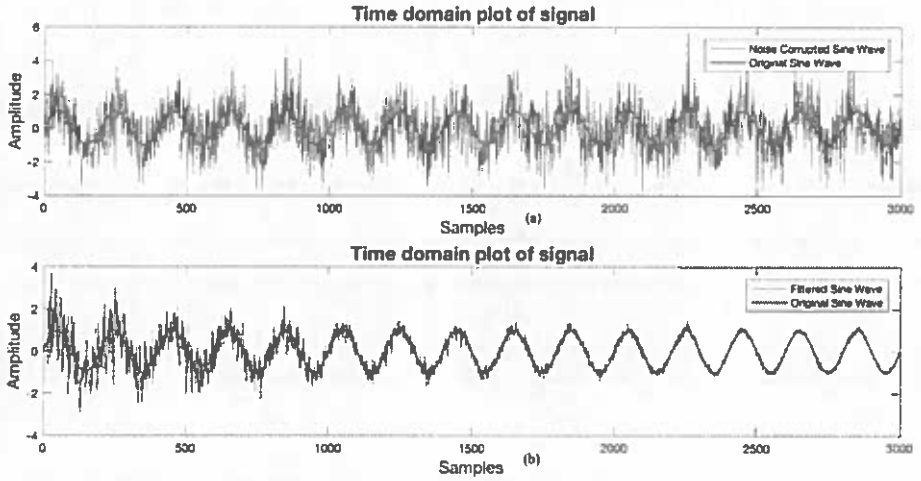


Figure 3.9 (a) Time domain plots of noise corrupted signal versus original signal; (b) Time domain plot of filtered sine wave using ANC vs original signal.

Table 3.4: Mean Square Prediction Errors (MSPE) for Adaptive Line Enhancer (ALE) versus Adaption Noise Cancellation (ANC)

Method	MSPE
ALE	0.15477
ANC	0.11615

d. **EEG Data Noise Suppression**: In Figure 1.16, it was seen that there is a strong noise component that occurs at 50 Hz. The time frequency analysis of this signal is seen in Figure 3.10 (a) and (b). It is seen that there is strong power component at 50 Hz corresponding to the noise. The goal is to eliminate this. By using a synthetic sine wave of frequency 50 Hz, an adaptive filter can be iterated to remove this noise component. The ANC algorithm was used to perform denoising of the of the EEG Signal. The results are seen in Figure 3.11 for different step sizes and different orders *M*.

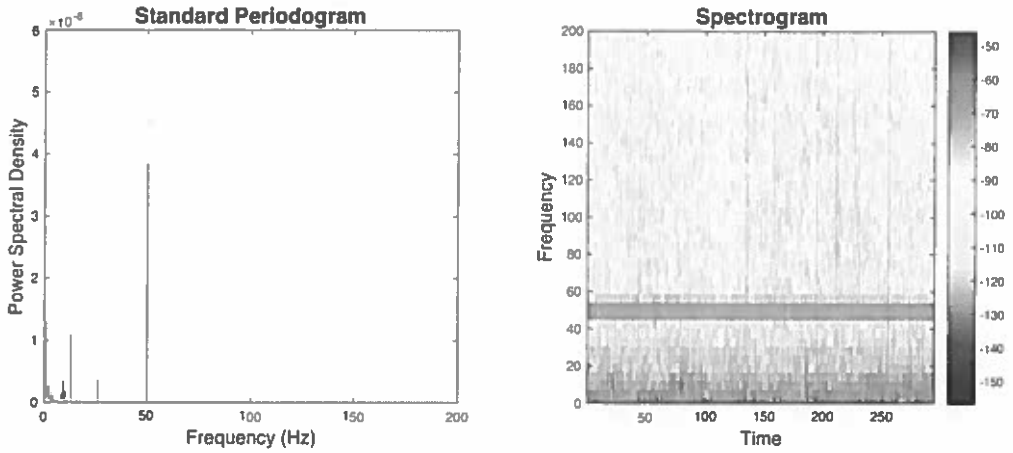


Figure 3.10 (a) Periodogram of EEG Signal; (b) Time – Frequency analysis using Spectrogram of EEG Signal

good showing of spectrograms for different parameters, taking care not to corrupt the signal

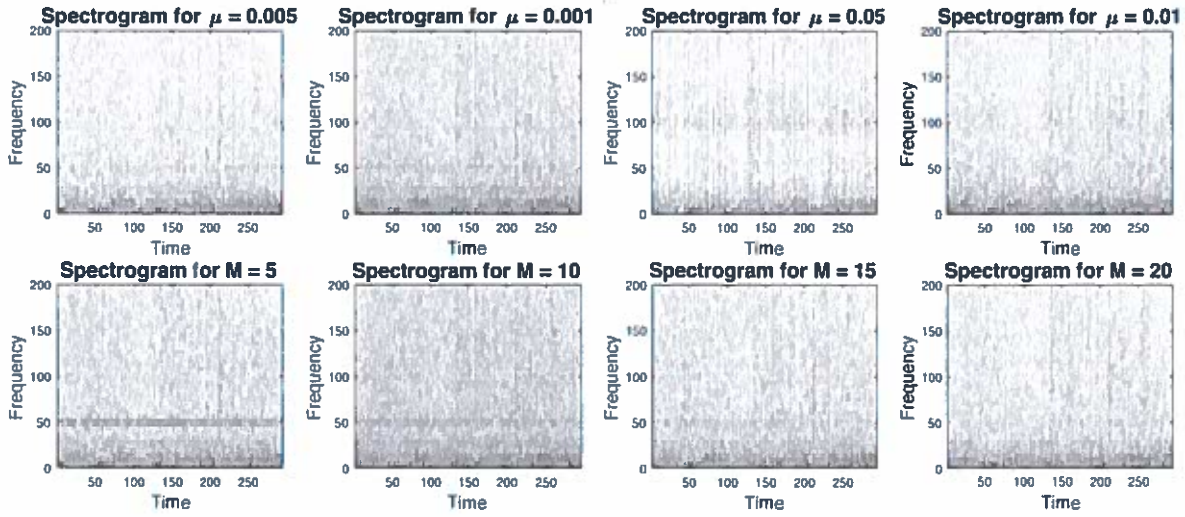


Figure 3.11 Spectrograms obtained after denoising using different input conditions

It is seen that the 50 Hz noise sample can be eliminated, provided the right order of the filter and step size is used. Care must also be taken so that components other than the 50 Hz sample are not eliminated. It is seen from Figure 3.11 that the step size of 0.05 performs well and completely removes the 50 Hz component, while preserving other signal components. A filter order of 15 performs well. Using a lower order filter does not completely eliminate the 50 Hz component, and using too high of an order like 20 removes some other frequencies from the spectrum. Thus denoising of any real-time signal can be performed using the ANC algorithm.

4. Widely Linear Filtering and Adaptive Spectrum Estimation

4.1 Complex LMS and Widely Linear Modelling

It is convenient to express real-time signals as complex data due for mathematical convenience. But this presents a challenge in terms of modifying an algorithm to suit complex data. The LMS algorithm extended to complex variants is known as the Complex LMS (CLMS). It follows a similar weight updating process represented in Section 3.1, but is linearly extended to cover the complex aspect. It can be summarized into:

$$\begin{aligned} \hat{y}(n) &= h^H(n)x(n) \\ e(n) &= y(n) - \hat{y}(n) \\ w(n+1) &= w(n) + \mu x(n)e^*(n) \end{aligned} \tag{21}$$

These equations are known to suit only circular data. To extend this to all complex data, an augmented input matix is used that uses both the instantaneous input and its complex conjugate for updating weights. This method is referred to as the Augmented Complex LMS (ACLMS). Two different weight matrices are used to reflect conjugate values. The equations are summarized as:

$$\begin{aligned} \hat{y}(n) &= h^H(n)x(n) + g^H(n)x(n) \\ e(n) &= y(n) - \hat{y}(n) \\ h(n+1) &= h(n) + \mu x(n)e^*(n) \\ g(n+1) &= g(n) + \mu x^*(n)e^*(n) \end{aligned} \tag{22}$$

a. **CLMS and ACLMS Implementation**: The CLMS and ACLMS were implemented for a **circular** white Gaussian noise. The results are consolidated in Figure 4.1 and 4.2. It is seen that the CLMS algorithm has a larger rate of convergence, is not very stable and also does not converge exactly to the required values. Whereas, the ACLMS possesses a better rate of convergence, is very stable and converges to the right values. The steady state errors are seen in Table 4.1. As expected, the ACLMS possesses a smaller steady state error.

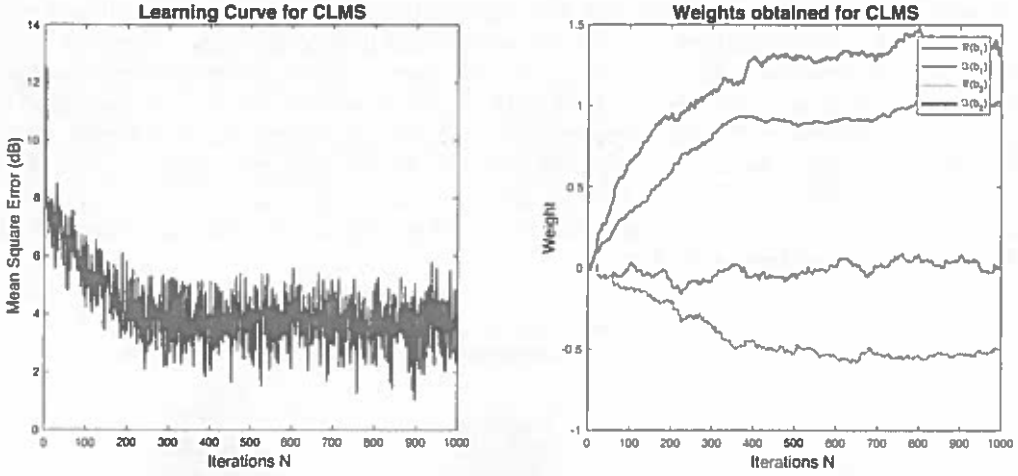


Figure 4.1 (a) Learning Curve obtained for the CLMS algorithm; (b) Weights obtained using CLMS. The Real and Imaginary components are seen here

Table 4.1 Steady State Errors Obtained for the CLMS and ACLMS methods

Method	Steady State Error
CLMS	2.3846
ACLMS	2.7395×10^{-5}

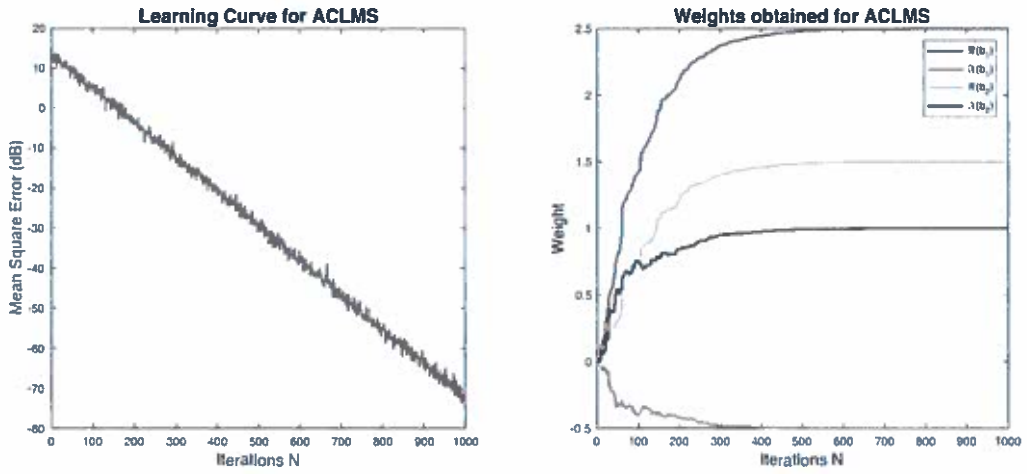


Figure 4.2 (a) Learning Curve obtained for the ACLMS algorithm; (b) Weights obtained using ACLMS. The Real and Imaginary components are seen here

Although the given input is circular, the ACLMS proves to be better as the two weight updation equations are better tuned to obtain the complex values for a particular filter order and step size. For a different input signal, the CLMS may prove to be better for a different order and step size. This superiority of an algorithm for a given signal can only be verified through experimentation, although ACLMS should produce better results in most cases.

b. **Bivariate Wind Data**: The wind data for three cases (low, medium and high speed) are used here. Their scatter plots are seen in Figure 4.3. It is seen that the low speed data is circular while the medium and high speed and more elliptical. A one step ahead prediction is performed for the wind data using the CLMS and ACLMS algorithms. As the CLMS algorithm is more suited to circular data, it is expected that the CLMS algorithm will perform better for the low speed wind data and not perform well for the medium and high speed cases. Figure 4.4 represents the performance of both the predictive algorithms. The plots showcase the mean steady state error versus the filter orders of the two algorithms. It is observed that the CLMS algorithm performs well for the low speed wind data, producing lesser steady state error. This was expected due to the circular nature of the data. For the medium speed data, the ACLMS algorithms performs well for all filter lengths. For the high speed data, the ACLMS algorithm performs better for lower filter orders and worsens as the order increases. It must be noted that in this case, the ACLMS performs the best for lower filter orders like 2 and 3. Thus, it is seen that the performance of a predictive algorithm depends on the circularity of the data.

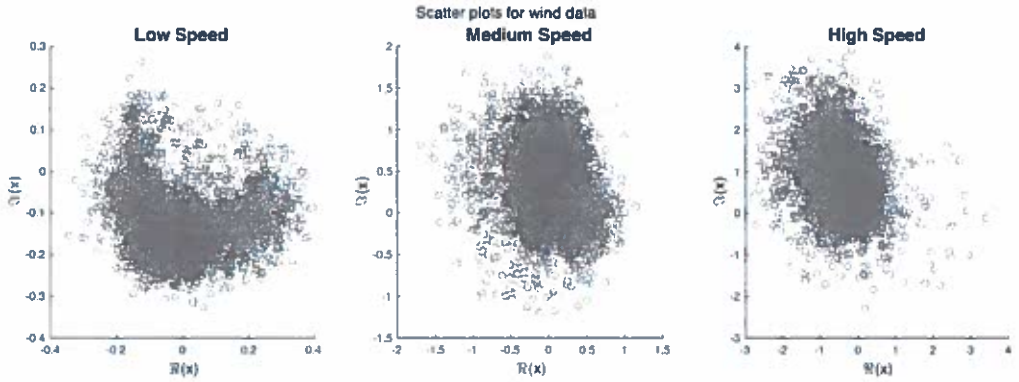


Figure 4.3 Scatter Plots obtained for different wind speeds.

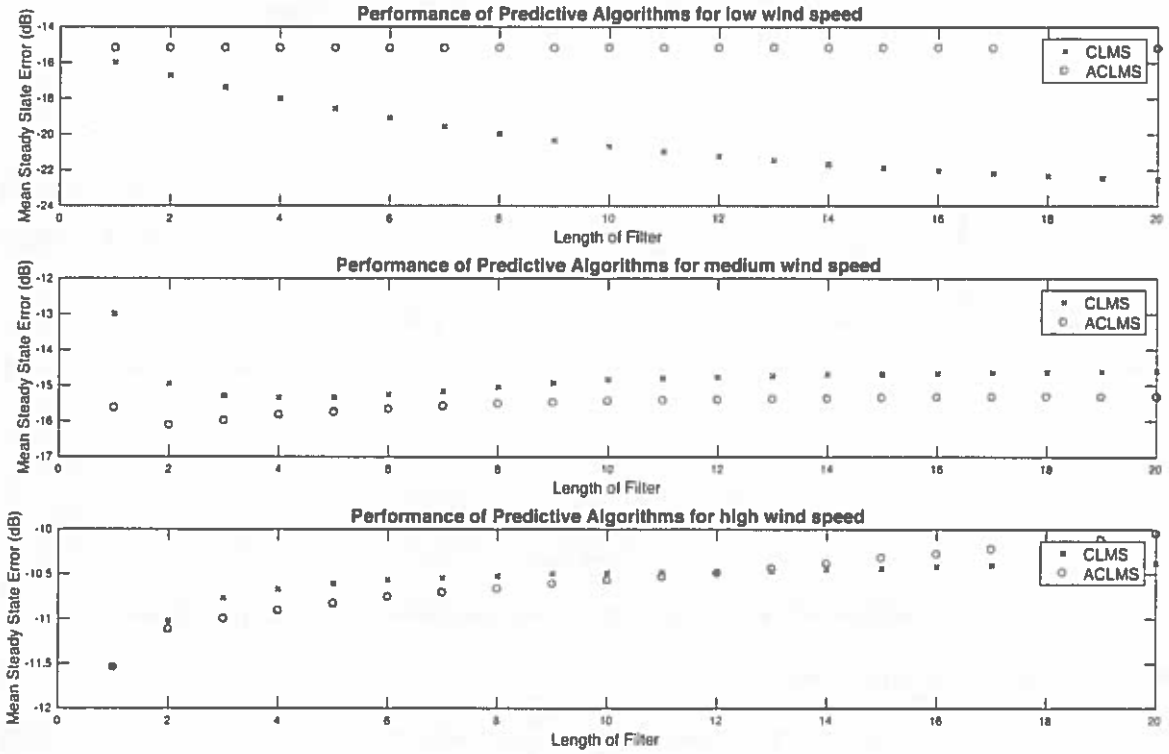


Figure 4.4 Performance of predictive algorithms for low, medium and high speed wind data.

Frequency Estimation in Three Phase Power Systems

c. The Clarke Voltages comprise of complex voltage vectors used for three phase power systems. By using the system of equations given in [6, 7], the Clarke Voltages for the balanced and unbalanced conditions are generated. The unbalanced condition is obtained by using three peak voltages of 0.9 V, 1 V and 1.1 V; while the phase differences are 0, $\pi/3$ and $\pi/2$. The circular plots are seen in Figure 4.5. It is seen that the balanced case represents a circular complex dataset, while the unbalanced case is non circular, typically being elliptical. The circularity of the non-circular case depends on the phase difference and peak voltages of the three sinusoidal components. It is observed that the balanced condition is a special case of the unbalanced scenario, when all the peak voltages are equal and the phase difference is null.

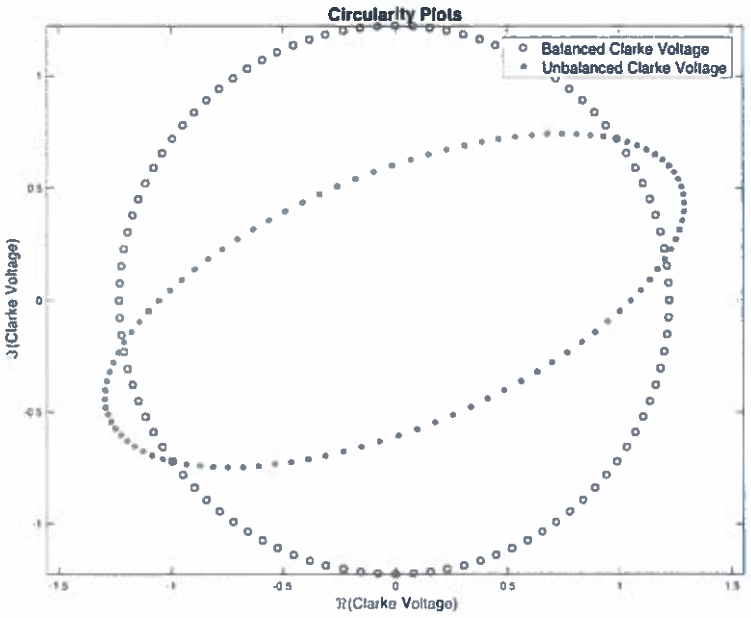
To identify a fault in the system, the circularity of a system is observed. If the plot is not circular, the difference in circularity with respect to an ideal balanced system can be determined, which in turn can provide information needed to adjust the voltage and phase differences. Thus, a fault can be identified and rectified using the circularity plots.

d. **Frequency of Balanced System**: The Strictly Linear updation equation is given as:

$$v(n+1) = h^*(n)v(n) \tag{23}$$

where

$$v(n) = \sqrt{\frac{3}{2}} V e^{j\{2\pi(f_0/f_s)n+\phi\}} \tag{24}$$



graph is too big, to occupy pages

Figure 4.5 Circularity plots of Clarke Voltages from a balanced and unbalanced three phase power system

Substituting Equation (24) in Equation (23):

$$\sqrt{\frac{3}{2}}Ve^{j\{2\pi(f_0/f_s)(n+1)+\phi\}} = h^*(n)\sqrt{\frac{3}{2}}Ve^{j\{2\pi(f_0/f_s)n+\phi\}} \tag{25}$$

Simplifying and rearranging:

$$f_0(n) = \frac{f_s}{2\pi}arctan\{\frac{-\Im\{h(n)\}}{\Re\{h(n)\}}\} \tag{26}$$

mathematical proof of frequency is missing

Frequency of Unbalanced System: The Widely Linear updation equation is given as:

$$v(n+1) = h^*(n)v(n) + g^*(n)v^*(n) \tag{27}$$

where

$$v(n) = A(n)e^{j\{2\pi(f_0/f_s)n+\phi\}} + B(n)e^{-j\{2\pi(f_0/f_s)n+\phi\}} \tag{28}$$

where $A(n)$ and $B(n)$ differ by a phase of $2\pi/3$. By replacing Equation (28) in (27), taking the difference between $A(n)$ and $B(n)$ and taking the absolute values of the real and imaginary components, the frequency is obtained as:

$$f_0(n) = \frac{f_s}{2\pi}arctan\{\frac{\sqrt{\Im^2\{h(n)\} - |g(n)|^2}}{\Re\{h(n)\}}\} \tag{29}$$

c. **Implementation of Predictive Algorithms:** The CLMS and ACLMS algorithms is applied to the balanced and unbalanced three phase systems generated in Section 3.1 (c). The nominal frequency used was 50 Hz (as in the UK). The frequency estimates obtained using the predictive algorithms are summarized in Table 4.2. It is seen that the ACLMS algorithm produces the right estimate in both the balanced and unbalanced cases, proving its robustness. Whereas, the CLMS algorithm produces the right estimate only in the balanced case and not in the balanced case. This can be attributed to the fact that the CLMS algorithm is not robust to non-circular data, as seen in Section 3.1 (b). Hence, the ACLMS algorithm is better to use to determine the Nominal Frequency in case no information about the input Clarke Voltage is provided.

Table 4.2 Nominal Frequencies obtained using the CLMS and ACLMS predictive algorithms.

Condition	CLMS	ACLMS
Balanced	50 Hz	50 Hz
Unbalanced	32.7994 Hz	50.0052 Hz

4.2 Adaptive AR Model Based Time-Frequency Estimation

a. **Power Spectrum**: As defined in [6], the frequency modulated signal is generated and its power spectrum is obtained. The original frequency signal and the power signal are seen in Figure 4.6. In Figure 4.6 (a), it is seen that the frequencies at samples 500 and 1000 are useful points as these contain changes in the frequency function. The frequencies at these two points are 100 and 350 respectively. These same points are observed in the power spectrum. No useful information is seen here. The power spectrum initially rises and falls, giving an indication that there may be two or three changes in frequency functions involved. However, it does not give any precise information capturing the changes in frequency. Thus, a different approach to study the frequency changes is needed.

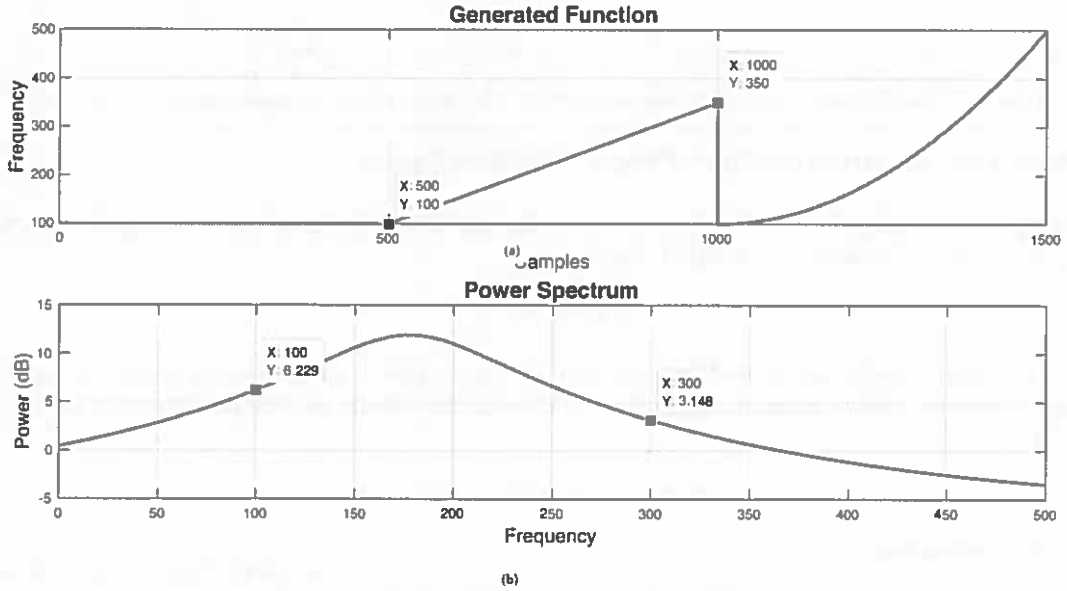


Figure 4.6 (a) Frequency function showing changes in frequency over time; (b) Power spectrum obtained from frequency modulated signal

b. **Time - Frequency Spectrum using Predictive Algorithm**: The CLMS algorithm is used to estimate the AR coefficient of the signal, by estimating the frequency at every instant of time. The spectrum obtained is seen in Figure 4.7. The figure is presented as a montage for comparison. The figure at the top represents the original frequency function and the figure at the bottom represents the time frequency spectrum. It is seen from the figure that the time frequency plot gives good information regarding the changes in frequency. It is seen that initially there is a constant power in between the samples 0 to 500, for a frequency value of 100 matching the original signal. In the linear variation of frequency between samples 500 to 1000, there is constant rise seen from the changing variations in red and black. At sample 1000, there is a sharp red indicating a drop in frequency value. Again the frequency rises, but at a sharper rate matching the input sequence. Thus, the time frequency plot is much more useful to obtain discernable properties with respect to changing frequencies in a spectrum.

unnecessary graph, appears it was placed to use space.

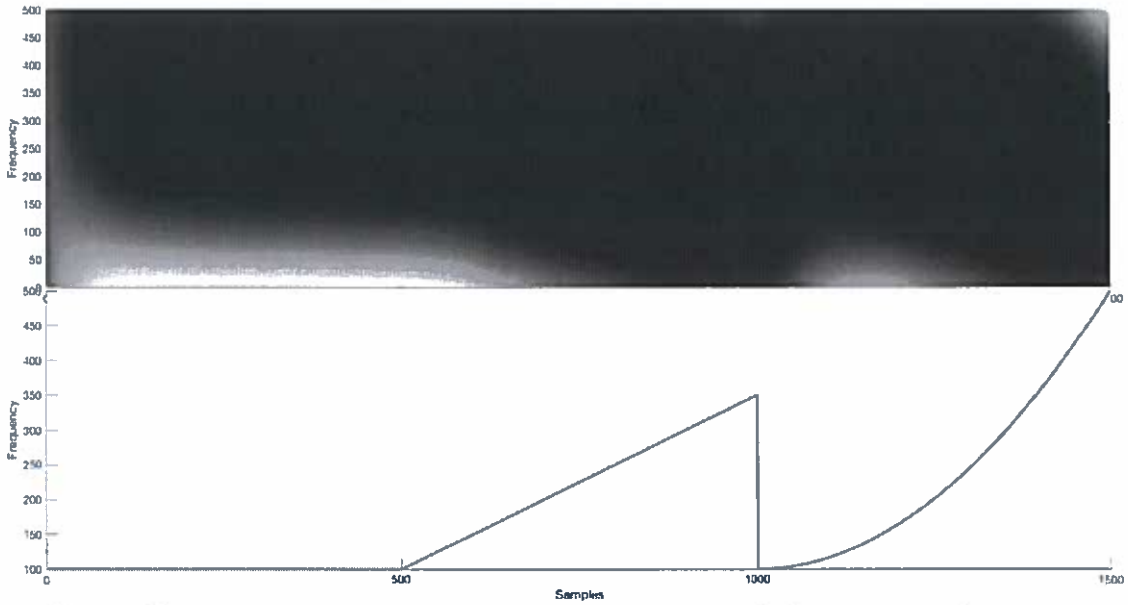


Figure 4.7 Time Frequency Spectrum obtained using the CLMS algorithm (Top); Original Frequency signal (Bottom)

4.3 A Real Time Spectrum Analyzer Using Least Mean Square

a. **Least Squares Solution:** A signal using a linear combination of harmonically related sinusoids is shown in [6]. The estimate $\hat{y}(n)$ can be written as a matrix equation as:

$$\hat{y}(n) = Fw \tag{30}$$

where F is a matrix containing all the harmonics and w is the unknown weights in the predictive algorithm. The least square solution aims to minimize the difference between the estimate and the actual signal $y(n)$. This can be represented as:

$$\min\|r\|^2 = \min|Fw - y|^2 \tag{31}$$

This can be expanded as:

not clear that F is a matrix.

$$\|r\|^2 = w^H F^H F w - 2y^H F w + y^H y \tag{32}$$

Taking the gradient and setting it to 0 for minima:

$$\begin{aligned} \nabla\|r\|^2 &= 2F^H F w - 2F^H y = 0 \\ &\Rightarrow F^H F w = F^H y \end{aligned} \tag{33}$$

We can observe from [6], that F is invertible. Thus:

$$w = (F^H F)^{-1} F^H y \tag{34}$$

The weights w obtained in Equation (34) is similar to the Fourier Transform Coefficients obtained from the Discrete Fourier Transform (DFT) as they are obtained for different sinusoidal harmonics of increasing power.

b. **Relation between DFT and basis function:** In this example, the basis function is considered to be an exponential function $e^{j2n\pi/N}$. The projection along any single direction can be obtained by taking Fw or $F(F^H F)^{-1} F^H y$. Thus, $F(F^H F)^{-1} F^H$ is considered to be the projection matrix. Hence, the basis function can be considered to be the exponential signal through which projections can be obtained. As the final expression of the sum of all projection vectors is similar to the Fourier Transform expression, the magnitude of the projections can be considered to Fourier

Coefficients. These correspond to the weights in the Equation (34). Thus, DFT can be considered to be an extension of the Least Square Interpretation problem.

c. **DFT-CLMS for Frequency Generated Signal**: The CLMS algorithm is applied to the obtain the DFT of the input signal used in Section 4.2 (a). By using an input phasor that contains harmonics of sinusoids, in the form of exponential functions of consecutive powers, the CLMS algorithms reciprocated a DFT function. Here, the weights obtained are similar to the Fourier Coefficients. Figure 4.8 represents the weight spectrum obtained for the sequence versus the original power signal obtained for the same. It is seen that the two plots are different as the weight spectrum calculates the frequency power for every instant of time, whereas the power spectrum is for an instantaneous frequency. Thus, a Time - Frequency analysis is needed to analyze a signal varying in time and frequency.

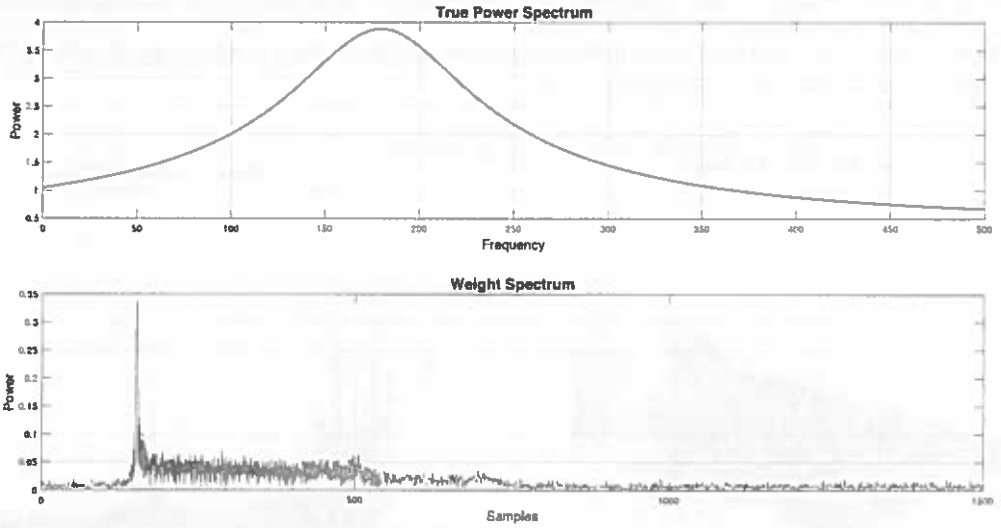


Figure 4.8 True Power Spectrum (Top) versus the Weight Spectrum (Bottom). The power spectrum is dependent on frequency whereas the weight spectrum is dependent on samples.

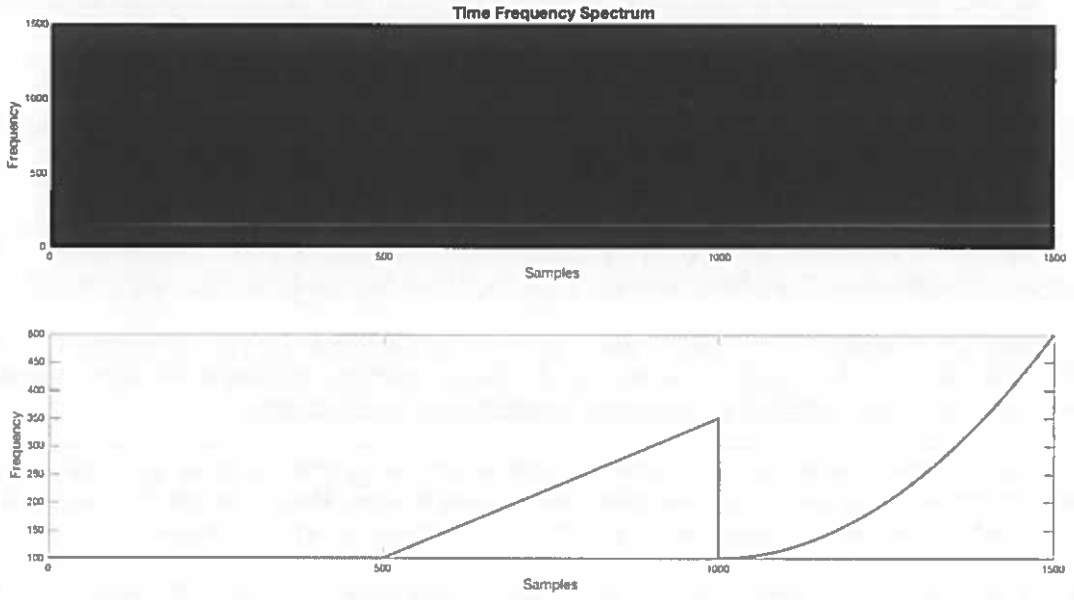


Figure 4.9 Time Frequency Spectrum obtained after DFT – CLMS (Top) versus original Frequency Spectrum (Bottom)

Figure 4.9 represents the Time - Frequency Spectrum obtained for the signal versus the original frequency used to generate the signal. It is seen that the DFT-CLMS representation is significantly better than the pure CLMS approach taken in Section 4.2 (b) and seen in Figure 4.7. The Frequency plot exactly follows the original frequency traversal, with a constant value from 0 to 500 samples, a linear variation from 500 to 1000 samples, and a quadratic function from 1000 to 1500 samples. This can be attributed to the complex harmonic phasor that is better able to obtain the weights needed for the process.

the limitations of the DFT-CLMS in the time frequency plot were not explored.

d. **DFT-CLMS for EEG Signal**: The DFT – CLMS approach is used to generate the frequency spectrum of the EEG Signal used in Section 1.4 (b). In that section, the periodogram was used to obtain the spectrum of frequencies present in the EEG Signal. Using the same approach described above, the Time - Frequency Spectrum is obtained for the EEG Signal. The 3-D Spectrum is seen in Figure 4.10 (a) and its 2-D counterpart extracted from the same plot, representing Power versus Frequency is seen in Figure 4.10 (b). It is seen that a similar analysis can be laid out to the signal, as done in Section 1.4 (b). In addition, the presence of the time variation of frequency presents more information that can be used to observe the variation of frequency during various instants of time. This makes the Time – Frequency Analysis a very important tool.

unorthodox representation of time frequency spectrum.

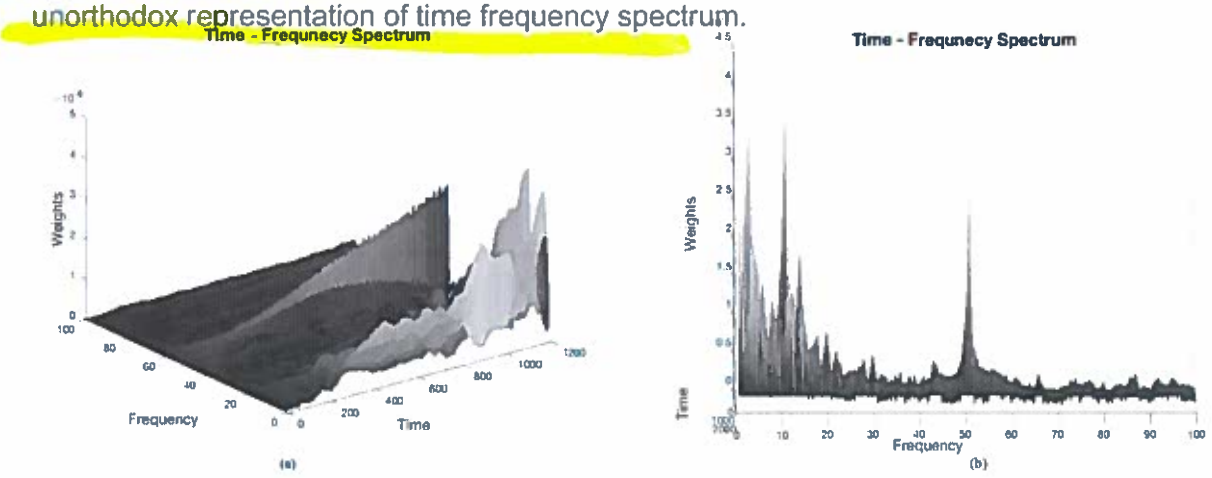


Figure 4.10 (a) Time Frequency Spectrum obtained after DFT – CLMS for EEG Signal (b) Weights versus Frequency for the EEG Signal

Conclusion

Important tools with respect to signal processing, machine learning and data analytics have been studied through the coursework

Section 1: The basics of Fourier Transforms and windows have been revisited, and their implementation to periodograms has been studied. Different properties of periodograms and spectrum estimation have been simulated and observed. The application of spectral estimation to real world data has also been successfully studied.

Section 2: Spectrum Analysis is seen in detail, and is applied to an Autoregressive Process. Principal Component Analysis is also studied for both feature reduction and denoising. Spectrum Analysis is also applied to real time Respiratory Sinus Arrhythmia (RSA) signals derived from practical experimentation.

Section 3: The Least Mean Square (LMS) algorithm is seen in detail along with variations including Leaky LMS, Gradient Adaptive Step Size LMS, and Generalized Normalized Gradient Descent Algorithms. Adaptive Noise Cancellation and Adaptive Line Enhancement as an extension of LMS is studied and simulated

Section 4: The Complex Least Mean Square (CLMS) and the Augmented Version of the same are studies for complex real time signals. The time frequency estimation and the DFT-CLMS algorithm are also studies and their indispensable uses are observed.

not necessary conclusion.. throughout the report space could have been used better with, for example, mathematical proofs.

References

[1] Mandic, D. P., "Adaptive Signal Processing and Machine Intelligence. Lecture 3: Spectrum Estimation", Imperial College London, 2017
[2] Hayes, M., "Statistical Signal Processing", John Wiley, 1996.
[3] Stoica, P., & Moses, R., "Spectral Analysis of Signals", Prentice Hall, 2005
[4] Mandic, D. P., "Adaptive Signal Processing and Machine Intelligence. Lecture 4: Modern Spectral Estimation", Imperial College London, 2017
[5] Mandic, D. P., "Adaptive Signal Processing and Machine Intelligence. Lecture: Linear Adaptive Filters and Applications", Imperial College London, 2017
[6] Mandic, D. P., "Adaptive Signal Processing and Machine Intelligence Coursework", Imperial College London, 2017
[7] Mandic, D. P., "Adaptive Signal Processing and Machine Intelligence. Lecture: Complex LMS", Imperial College London, 2017